

DaTra: Data Transformation Semantics as a Symmetric Monoidal Category on Atlas Presheaves

@qualiascript

ChatGPT

1 Dominions

Definition 1.1 (The Category of Dominions). The **Category of Dominions**, denoted $\mathbf{Dom}$, has as its objects sets whose cardinality is at most ω_0 , equivalently sets admitting an injection into ω_0 , and as its morphisms set-theoretic functions.

Definition 1.2 (The Domanial Embedding Functor). The **Domanial Embedding Functor**, denoted $\mathbf{Emb} : \mathbf{Dom} \rightarrow \mathbf{Set}$, sends each dominion to its underlying set.

2 Domanial Insertions

Definition 2.1 (The Category of Domanial Insertions). The **Category of Domanial Insertions**, denoted $\mathbf{DomIns}$, has dominions as objects and monomorphisms in $\mathbf{Dom}$ as morphisms.

Definition 2.2 (The Category of Codomanial Insertions). The **Category of Codomanial Insertions**, denoted $\mathbf{CoDomIns}$, is the opposite category $\mathbf{DomIns}^{\text{op}}$.

3 Chains

Definition 3.1 (Chain). A **Chain** is a totally ordered thin category, indexed skeletally by an ordinal strictly smaller than $\omega_0^{\omega_0}$.

Definition 3.2 (The Spine). The **spine** is the chain with ω_0 objects.

4 Consolidations

Definition 4.1 (The Category of Consolidations). The **Category of Consolidations**, denoted $\mathbf{Con}$, has chains as objects and point-surjective functors between their thin categories as morphisms.

Definition 4.2 (The Category of Coconsolidations). The **Category of Coconsolidations**, denoted $\mathbf{CoCon}$, is the opposite category $\mathbf{Con}^{\text{op}}$.

5 Transportation and Folios

Definition 5.1 (The Transportation Functor). The **Transportation Functor**, denoted $\mathbf{Tra} : \mathbf{Con} \rightarrow \mathbf{Set}$, sends a chain to its object set and a consolidation to its underlying set-theoretic surjection.

Definition 5.2 (Folio). A **folio** is a functor $F : S \rightarrow \mathbf{CoCon}$, where S is the spine, such that $F(0)$ is the singleton chain.

6 Paginations

Definition 6.1 (The Category of Paginations). The **Category of Paginations**, denoted $\mathbf{Pag}$, has as objects pairs (H, E) where $H = \mathbf{Tra} \circ F^{\text{op}}$ for a folio F , and E is the category of elements of H . For $W : \mathbf{Pag}$, write W_H for the first inclusion and W_E for the second inclusion. A morphism $T : X \rightarrow Y$ in $\mathbf{Pag}$ is a functor $T : X_E \rightarrow Y_E$. Thus, for every morphism $f : x \rightarrow y$ of X_E , the following square commutes:

$$\begin{array}{ccc} x = (m, i) & \xrightarrow{f} & (n, X_H(f)(i)) \\ T \downarrow & & \downarrow T \\ (m', i') & \xrightarrow{T(f)} & (n', Y_H(T(f))(i')). \end{array}$$

7 Atlases

Definition 7.1 (The Category of Atlases). The **Category of Atlases**, denoted $\mathbf{Atl}$, has as objects pairs (P, G) , where $P : \mathbf{Pag}$ and $G : P_E \rightarrow \mathbf{DomIns}$ is a functor, equivalently a presheaf $G : P_E^{\text{op}} \rightarrow \mathbf{CoDomIns}$. For $P_H : C \rightarrow \mathbf{Set}$, $m \in |C|$, and distinct elements $k, k' \in |P_H(m)|$, the pullback in G of the arrows induced by $(m \rightarrow 0)(k)$ and $(m \rightarrow 0)(k')$ is empty.

There is an integer w such that, for every $w' > w$, $P_H(w' \rightarrow w) = \text{Id}$, and, for every $k \in |P_H(w)|$, the corresponding arrow $G((w' \rightarrow w)(k))$ is the identity. The least such integer is the **cardinality of the atlas**, denoted $|A|$.

For $X : \mathbf{Atl}$, write X_P and X_G for the two components and put $X_H = (X_P)_H$ and $X_E = (X_P)_E$. A morphism $T : X \rightarrow Y$ is a pair (T_P, T_A) , where $T_P : X_P \rightarrow Y_P$ is a pagination morphism and $T_A : X_G \Rightarrow Y_G \circ T_E$ is a natural transformation. Thus, for every $f : x \rightarrow y$ in X_E , the following square commutes:

$$\begin{array}{ccc} X_G(x) & \xrightarrow{X_G(f)} & X_G(y) \\ (T_A)_x \downarrow & & \downarrow (T_A)_y \\ Y_G(T_E(x)) & \xrightarrow{Y_G(T_E(f))} & Y_G(T_E(y)). \end{array}$$

Finally, if $x = |X|$, $x' > x$, and $r \in |X_H(|X| - 1)|$, then $T_P(x', r) = T_P(x, r)$ and $(T_A)_{(x', r)} = (T_A)_{(x, r)}$.

Definition 7.2 (Cardinality of an Atlas). The **cardinality** $|A|$ of an atlas is the least positive integer after which its spine repeats the final genuine page $|A| - 1$.

Definition 7.3 (Extent of an Atlas). The **extent** of A is $\text{Ex}(A) = A_G(0, 0)$. Since objects of $\mathbf{DomIns}$ are dominions, $\text{Ex}(A) : \mathbf{Dom}$ for every $A : \mathbf{Atl}$.

Definition 7.4 (Territory of an Atlas). Let $M = A_H(|A| - 1)$. The **territory** is the indexed family $\text{Ter}(A) : M \rightarrow \mathbf{Dom}$ given by $\text{Ter}(A)(k) = A_G(|A| - 1, k)$.

Definition 7.5 (n th Region of an Atlas). The **n th region** of A is $\text{Ter}(A)(n)$, with indexing beginning at 0.

8 Transposals and Traversals

Definition 8.1 (The Category of Atlas Transposals). The **Category of Atlas Transposals**, denoted $\mathbf{AtlTrap}$, is the wide subcategory of $\mathbf{Atl}$ whose morphisms $F : X \rightarrow Y$ have point-injective object maps F_E .

Definition 8.2 (The Category of Atlas Traversals). The **Category of Atlas Traversals**, denoted AtlTrav , is the wide subcategory of AtlTrap whose morphisms satisfy the following conditions. For $F : X \rightarrow Y$, let $x = (m, i)$ and $y = (m', j)$, put $p = \min(m, m')$, write $F_E(x) = (n, i')$ and $F_E(y) = (n', j')$, and put $p' = \min(n, n')$. If

$$X_H(m \rightarrow p)(i) < X_H(m' \rightarrow p)(j),$$

then

$$Y_H(n \rightarrow p')(i') < Y_H(n' \rightarrow p')(j').$$

Furthermore, if $q \in |X|$ and $t \in X_G(q)$ is in the image of a final region—that is, there exist $k \in |\text{Ter}(X)|$ and $l \in \text{Ter}(X)(k)$ whose image under $X_G(|X| - 1 \rightarrow q)(k)$ is t —then this property is preserved by F .

Definition 8.3 (The Category of Stable Atlas Traversals). The **Category of Stable Atlas Traversals**, denoted AtlTras , is the wide subcategory of AtlTrav whose morphisms $F : X \rightarrow Y$ satisfy $F_E(0, 0) = (0, 0)$.

9 Maps and Cartography

Definition 9.1 (The Category of Atlas Maps). The **Category of Atlas Maps**, denoted AtlMap , is the full subcategory of atlases for which every element of the extent is in the image of a final region.

Definition 9.2 (The Atlas Map Inclusion Functor). The **Atlas Map Inclusion Functor** $\text{AtlMapInc} : \text{AtlMap} \rightarrow \text{Atl}$ is the canonical inclusion.

Definition 9.3 (The Category of Atlas Traversal Maps). The **Category of Atlas Traversal Maps**, denoted AtlTravMap , is the full subcategory of AtlTrav whose objects are Atlas Maps.

Definition 9.4 (The Atlas Traversal Map Inclusion Functor). The canonical inclusion is denoted $\text{AtlTravMapInc} : \text{AtlTravMap} \rightarrow \text{AtlTrav}$.

Lemma 9.5 (Cartography Lemma). *The inclusion AtlTravMapInc has a right adjoint, denoted the **Charting Functor** $\text{Chr} : \text{AtlTrav} \rightarrow \text{AtlTravMap}$.*

Proof sketch. On an object $X : \text{AtlTrav}$, the functor Chr sends X to the universal arrow from AtlTravMapInc to X . Write $X' = \text{AtlTravMapInc}(\text{Chr}(X))$, together with $H : X' \rightarrow X$, with X' terminal among objects having this property. The solution is the DaTra map satisfying $\text{Ter}(X') \cong \text{Ter}(X)$. It is unique because AtlTrav preserves orders and terminal because H_E is faithful.

10 Coalitions

Definition 10.1 (The Coalizing Functor). The **Coalizing Functor**, denoted $\text{Coa} : \text{AtlTras} \rightarrow \text{DomIns}$, is the extent of the charted atlas: $\text{Coa} = \text{Ex} \circ \text{Chr}$.

Definition 10.2 (The Coalition of an Atlas). The **coalition** of an atlas X is the extent of its chart, regarding X as an object of the wide category of stable traversals.

Definition 10.3 (The Domanial Inclusion Functor). The **Domanial Inclusion Functor** $\text{DomInc} : \text{DomIns} \rightarrow \text{AtlTras}$ sends a dominion to the atlas of cardinality 1 whose coalition is that dominion. It is left adjoint to Coa : for $X : \text{DomIns}$ and $Y : \text{AtlTras}$, naturally in X and Y ,

$$\text{Hom}_{\text{AtlTras}}(\text{DomInc}(X), Y) \cong \text{Hom}_{\text{DomIns}}(X, \text{Coa}(Y)),$$

and $\text{Coa}(\text{DomInc}(X)) \cong X$.

11 Data Transformations

Definition 11.1 (The Category of Data Transformations). The **Category of Data Transformations**, also called the category of **Data Transformation Sets** or **DaTra Sets**, is the presheaf category

$$\text{DaTra} = [\text{Atl}^{\text{op}}, \text{Set}].$$

As a presheaf category, it is a topos.

Definition 11.2 (Navigation). A **navigation** of $D : \text{DaTra}$ is a monomorphism $\text{Nav} : \text{Yo}(A) \rightarrow D$ for some atlas A .

Definition 11.3 (Expedition). An **expedition** is a navigation represented by an Atlas Map.

Definition 11.4 (DaTra Restriction). For a wide subcategory W of Atl , its **DaTra restriction** is the wide subcategory whose arrows $F : X \rightarrow Y$ have this property: pulling F back along any navigation $\text{Yo}(A) \rightarrow Y$ is either empty, or is represented by a navigation $\text{Yo}(B) \rightarrow X$ and an arrow $f : B \rightarrow A$ in W . In the represented case the square

$$\begin{array}{ccc} \text{Yo}(B) & \xrightarrow{\text{Nav}'} & X \\ \text{Yo}(f) \downarrow & & \downarrow F \\ \text{Yo}(A) & \xrightarrow{\text{Nav}} & Y \end{array}$$

is a pullback.

Definition 11.5 (The Category of Data Transposals). The **Category of Data Transposals**, denoted DaTraP , is the DaTra restriction on AtlTraP .

Definition 11.6 (The Category of Data Traversals). The **Category of Data Traversals**, denoted DaTraV , is the DaTra restriction on AtlTraV .

Definition 11.7 (The Category of Stable Data Traversals). The **Category of Stable Data Traversals**, denoted DaTraS , is the DaTra restriction on AtlTraS .

Definition 11.8 (Data Transformation Maps). The **Category of Data Transformation Maps**, denoted DaTraMap , is the full subcategory of DaTra Sets all of whose navigations are expeditions.

12 Horizontal Atlases

Definition 12.1 (The Empty Atlas). The **Empty Atlas** is $\text{AtlI} = \text{DomInc}(0)$.

Definition 12.2 (The Atlas Horizontal Sum Bifunctor). The **Atlas Horizontal Sum Bifunctor**, denoted $\text{AtlHorSum} : \text{Atl} \times \text{Atl} \rightarrow \text{Atl}$, is defined as follows. Given two morphisms $F : X \rightarrow Y$ and $F' : X' \rightarrow Y'$ in Atl , if $F = \text{AtlI}$, then $\text{AtlHorSum}(F, F') = F'$, and if $F' = \text{AtlI}$, then $\text{AtlHorSum}(F, F') = F$. Otherwise, let $G : K \rightarrow R$ be the universal arrow from Atl to $R = Y \sqcup Y'$ such that there exist morphisms $M : X \rightarrow K$ and $M' : X' \rightarrow K$ in AtlTraV satisfying $M_E(0, 0) = (1, 0)$ and $M'_E(0, 0) = (1, 1)$. For the injections $Y_i : Y \rightarrow R$ and $Y'_i : Y' \rightarrow R$, these satisfy $G \circ M \cong Y_i \circ F$ and $G \circ M' \cong Y'_i \circ F'$. Finally, $\text{AtlHorSum}(F, F')$ is obtained by precomposing G with origin $X \sqcup X'$ in AtlTraP .

Lemma 12.3 (Horizontal Lemma). *The Atlas Horizontal Sum Bifunctor AtlHorSum exists.*

Proof sketch. The cases $F = \text{AtlI}$ or $F' = \text{AtlI}$ are trivial. Otherwise, for $F : X \rightarrow Y$ and $F' : X' \rightarrow Y'$, let $T : \text{Atl}$, let $R = Y \sqcup Y'$, and let $G : T \rightarrow R$, where

$$|T| = \max(|X|, |X'|) + 1.$$

Set $T_G(0, 0) = \text{Ex}(X) \sqcup \text{Ex}(X')$, $R_G(0, 0) = \text{Ex}(Y) \sqcup \text{Ex}(Y')$, and let $G_{A,(0,0)}$ be the induced monomorphism. For each positive integer m , set $T_H(m) = X_H(m-1) \sqcup X'_H(m-1)$. For $n \in |A_P(m)|$, if $n < M(m)$, put

$$T_G(m, n) = X_G(m-1, n), \quad G_{A,(m,n)} = F_{A,(m-1,n)};$$

otherwise put

$$T_G(m, n) = X'_G(m-1, n - |X_H(m-1)|),$$

and

$$G_{A,(m,n)} = F'_{A,(m-1,n-|X_H(m-1)|)}.$$

This defines G , which is initial among solutions.

Definition 12.4 (The Atlas Braider). The **Atlas Braider** $\text{AtlBrd}_{F,F'} : \text{AtlHorSum}(F, F') \rightarrow \text{AtlHorSum}(F', F)$ is defined as follows. If $F = \text{Atll}$ or $F' = \text{Atll}$, then $\text{AtlBrd}_{F,F'} = \text{Id}$. Otherwise, let $R = \text{AtlHorSum}(F, F')$, and let $G : H \rightarrow R$ be the universal arrow from AtlTrap to R satisfying $G_E(1, 0) = (1, 1)$ and $G_E(1, 1) = (1, 0)$. Then $\text{AtlBrd}_{F,F'} = G^{-1}$, and $\text{AtlBrd}_{F,F'} \circ \text{AtlBrd}_{F',F} = \text{Id}$.

Definition 12.5 (The Atlas Associator). The **Atlas Associator**, denoted

$$\text{AtlAsoc}_{F,F',F''} : \text{AtlHorSum}(\text{AtlHorSum}(F, F'), F'') \longrightarrow \text{AtlHorSum}(F, \text{AtlHorSum}(F', F'')),$$

is defined as follows. If $F = \text{Atll}$, $F' = \text{Atll}$, or $F'' = \text{Atll}$, then $\text{AtlAsoc}_{F,F',F''} = \text{Id}$. Otherwise, let $R = \text{AtlHorSum}(\text{AtlHorSum}(F, F'), F'')$, and let $G : H \rightarrow R$ be the universal arrow from AtlTrap to R such that $G_E(2, 0) = (1, 0)$. Suppose also that there exists a morphism $G' : \text{AtlHorSum}(F', F'') \rightarrow H$ in AtlTrav satisfying $G'_E(0, 0) = (1, 1)$. The inverse of G is analogous. Finally, AtlAsoc is obtained by flattening G .

Definition 12.6 (The Atlas Unitors). The **Atlas Left Unitor** $\text{Lu} : \text{AtlHorSum}(\text{Atll}, F) \rightarrow F$ and the **Atlas Right Unitor** $\text{Ru} : \text{AtlHorSum}(F, \text{Atll}) \rightarrow F$ are given by $\text{Lu} = \text{Id}$ and $\text{Ru} = \text{Id}$.

13 Horizontal Data Traversals

Definition 13.1 (The Empty Map). The **Empty Map**, denoted I , is the Yoneda embedding of the Empty Atlas:

$$I = \text{Yo}(\text{Atll}).$$

Definition 13.2 (The Horizontal Sum Bifunctor). The **Horizontal Sum Bifunctor**, denoted $\text{HorSum} : \text{DaTrav} \times \text{DaTrav} \rightarrow \text{DaTrav}$, or in infix notation by $+_{<} : \text{DaTrav} \times \text{DaTrav} \rightarrow \text{DaTrav}$, is the Day convolution extension of AtlHorSum .

Definition 13.3 (The Data Traversal Braider). The **Data Traversal Braider** $\text{Brd}_{F,F'} : F +_{<} F' \rightarrow F' +_{<} F$ is the Day convolution extension of AtlBrd .

Definition 13.4 (The Data Traversal Associator). The **Data Traversal Associator**

$$\text{Asoc}_{F,F',F''} : (F +_{<} F') +_{<} F'' \longrightarrow F +_{<} (F' +_{<} F'')$$

is the Day convolution extension of AtlAsoc .

Definition 13.5 (The Data Traversal Unitors). The **Data Traversal Left Unitor** $\text{Lu} : I +_{<} F \rightarrow F$ and the **Data Traversal Right Unitor** $\text{Ru} : F +_{<} I \rightarrow F$ are the Day convolution extensions of AtlLu and AtlRu .

Definition 13.6 (The Data Traversal Monoidal Category). The **Data Traversal Monoidal Category**, denoted DaTravMon , is

$$\text{DaTravMon} = (\text{DaTrav}, +_{<}, I).$$

The coherence conditions are checked by observing that Brd , Asoc , Lu , and Ru are sent to isomorphisms in DaTra .

A Lean Formalization

The following is an ASCII rendering of the complete Lean source corresponding to the mathematical development above. It replaces Lean’s conventional Unicode surface notation solely for portable typesetting; the checked source file remains `datra.lean`.

```
1  import Mathlib
2
3  namespace Datra
4
5  open CategoryTheory CategoryTheory.Functor CategoryTheory.Limits Opposite
6  open scoped Ordinal
7
8  universe u
9
10 noncomputable section
11
12 /-- A dominion is a set equipped with a certificate of countability. -/
13 structure Dominion where
14   Carrier : Type
15   rank : Function.Embedding Carrier Nat
16
17 abbrev Dom := Dominion
18
19 instance : CoeSort Dominion Type where coe X := X.Carrier
20
21 instance : Category Dominion where
22   Hom X Y := X -> Y
23   id _ := id
24   comp f g := g o f
25   id_comp _ := rfl
26   comp_id _ := rfl
27   assoc _ _ _ := rfl
28
29 def Emb : Dom ==> Type where
30   obj X := X
31   map f := f
32
33 structure DomIns where
34   toDom : Dom
35
36 instance : CoeSort DomIns Type where coe X := X.toDom.Carrier
37
38 instance : Category DomIns where
39   Hom X Y := Function.Embedding X Y
40   id X := Function.Embedding.refl X
41   comp f g := f.trans g
42   id_comp f := by ext; rfl
43   comp_id f := by ext; rfl
44   assoc f g h := by ext; rfl
45
46 instance {X Y : DomIns} : CoeFun (X --> Y) (fun _ => X -> Y) where
47   coe f := f.toFun
48
49 @[ext]
50 theorem DomIns.hom_ext {X Y : DomIns} (f g : X --> Y)
51   (h : forall x, f x = g x) : f = g := by
52   exact Function.Embedding.ext h
53
54 instance {X Y : DomIns} (f : X --> Y) : Mono f where
55   right_cancellation g h w := by
56     apply DomIns.hom_ext
57     intro x
58     apply f.injective
59     exact congrFun (congrArg Function.Embedding.toFun w) x
60
61 def DomIns.forget : DomIns ==> Dom where
62   obj X := X.toDom
63   map f := f.toFun
64
65 abbrev CoDomIns := Opposite DomIns
66
```

```

67 /-- A chain carries its skeletal well-ordered object type and a proof that
68 its order type lies below  $\omega_0$ . Retaining the carrier explicitly makes
69 ordinal sums of chains definitionally usable. -/
70 structure Chain where
71   Obj : Type
72   linearOrder : LinearOrder Obj
73   wellFoundedLT : WellFoundedLT Obj
74   orderType_lt : Ordinal.type (fun x y : Obj => x < y) <
75     Ordinal.omega0 ^ Ordinal.omega0
76
77 attribute [instance] Chain.linearOrder Chain.wellFoundedLT
78
79 def Chain.sum (X Y : Chain) : Chain where
80   Obj := Lex (Sum X.Obj Y.Obj)
81   linearOrder := inferInstance
82   wellFoundedLT := <|by
83     change WellFounded (Sum.Lex (fun x y : X.Obj => x < y)
84       (fun x y : Y.Obj => x < y))
85     exact Sum.lex_wf wellFounded_lt wellFounded_lt|>
86   orderType_lt := by
87     change Ordinal.type (Sum.Lex (fun x y : X.Obj => x < y)
88       (fun x y : Y.Obj => x < y)) < Ordinal.omega0 ^ Ordinal.omega0
89   rw [Ordinal.type_sum_lex]
90   exact Ordinal.principal_add_omega0_opow Ordinal.omega0
91     X.orderType_lt Y.orderType_lt
92
93 /-- The common page-indexing chain. -/
94 def Spine : Chain where
95   Obj := Nat
96   linearOrder := inferInstance
97   wellFoundedLT := inferInstance
98   orderType_lt := by
99     have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
100       Ordinal.omega0 ^ Ordinal.omega0 :=
101       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
102       Ordinal.one_lt_omega0
103     change typeLT Nat < Ordinal.omega0 ^ Ordinal.omega0
104     rw [Ordinal.type_nat_lt]
105     simpa only [Ordinal.opow_one] using hpow
106
107 abbrev Con := Chain
108
109 /-- A functor between ordinal chains is equivalently a monotone object map. -/
110 structure ConHom (X Y : Con) where
111   toFun : X.Obj -> Y.Obj
112   monotone : Monotone toFun
113   point_surjective : Function.Surjective toFun
114
115 def ConHom.sum {X X' Y Y' : Con} (f : ConHom X Y) (g : ConHom X' Y') :
116   ConHom (Chain.sum X X') (Chain.sum Y Y') where
117   toFun z := toLex (Sum.map f.toFun g.toFun (ofLex z))
118   monotone := by
119     intro a b h
120     change Sum.Lex (fun x y : X.Obj => x <= y) (fun x y : X'.Obj => x <= y)
121       (ofLex a) (ofLex b) at h
122     change Sum.Lex (fun x y : Y.Obj => x <= y) (fun x y : Y'.Obj => x <= y)
123       (Sum.map f.toFun g.toFun (ofLex a))
124       (Sum.map f.toFun g.toFun (ofLex b))
125     generalize ha : ofLex a = sa at h | -
126     generalize hb : ofLex b = sb at h | -
127     rcases sa with x | x <; rcases sb with y | y
128     .
129     simp only [Sum.map]
130     exact Sum.Lex.inl (f.monotone (Sum.lex_inl_inl.mp h))
131     . simp only [Sum.map]
132     exact Sum.Lex.sep _ _
133     . exact (Sum.lex_inr_inl h).elim
134     .
135     simp only [Sum.map]
136     exact Sum.Lex.inr (g.monotone (Sum.lex_inr_inr.mp h))
137   point_surjective := by
138     intro z
139     rcases hz : ofLex z with y | y

```

```

140   . obtain <|x, rfl|> := f.point_surjective y
141   refine <|toLex (Sum.inl x), ?_|>
142   apply toLex.injective
143   simp using hz.symm
144   . obtain <|x, rfl|> := g.point_surjective y
145   refine <|toLex (Sum.inr x), ?_|>
146   apply toLex.injective
147   simp using hz.symm
148
149 instance {X Y : Con} : CoeFun (ConHom X Y) (fun _ => X.Obj -> Y.Obj) where
150   coe f := f.toFun
151
152 @[ext]
153 theorem ConHom.ext {X Y : Con} (f g : ConHom X Y)
154   (h : forall x, f x = g x) : f = g := by
155   cases f
156   cases g
157   simp_all only [mk.injEq]
158   exact funext h
159
160 instance : Category.{0} Con where
161   Hom := ConHom
162   id X := <|id, monotone_id, Function.surjective_id|>
163   comp f g := <|g o f, g.monotone.comp f.monotone,
164     g.point_surjective.comp f.point_surjective|>
165   id_comp f := by ext; rfl
166   comp_id f := by ext; rfl
167   assoc f g h := by ext; rfl
168
169 instance {X Y : Con} : CoeFun (X --> Y) (fun _ => X.Obj -> Y.Obj) where
170   coe f := f.toFun
171
172 def Con.asFunctor {X Y : Con} (f : X --> Y) : X.Obj ==> Y.Obj where
173   obj := f
174   map g := homOfLE (f.monotone (leOfHom g))
175
176 abbrev CoCon := Opposite Con
177
178 def Tra : Con ==> Type where
179   obj X := X.Obj
180   map f := f.toFun
181
182 /-- A folio is stored by the least finite presentation of its eventually
183 constant spine functor. `length` is its cardinality: page `length - 1` is
184 the final genuine page and every later spine page repeats it. -/
185 structure Folio where
186   length : Nat
187   positive : 0 < length
188   core : Functor.{0, 0} (Fin length) CoCon
189   originEquiv : (core.obj <|0, positive|>).unop.Obj Equiv Unit
190
191 def Folio.paddedIndex (W : Folio) (n : Nat) : Fin W.length :=
192   <|min n (W.length - 1), lt_of_le_of_lt (min_le_right _ _)
193     (Nat.sub_lt W.positive (by omega))|>
194
195 theorem Folio.paddedIndex_mono (W : Folio) :
196   Monotone W.paddedIndex := by
197   intro a b hab
198   exact min_le_min hab le_rfl
199
200 def Folio.spineBase (W : Folio) : Nat ==> Fin W.length where
201   obj n := W.paddedIndex n
202   map f := homOfLE (W.paddedIndex_mono (leOfHom f))
203   map_id _ := Subsingleton.elim _ _
204   map_comp _ _ := Subsingleton.elim _ _
205
206 /-- The actual functor on the spine denoted by the finite presentation. -/
207 def Folio.F (W : Folio) : Nat ==> CoCon := W.spineBase then W.core
208
209 /-- The page functor on the opposite spine. -/
210 def Folio.spineH (W : Folio) : Natop ==> Type := W.F.leftOp then Tra
211
212 def Folio.H (W : Folio) : (Fin W.length)op ==> Type := W.core.leftOp then Tra

```



```

213
214 def Folio.E (W : Folio) : Type 0 := W.H.Elements
215
216 instance (W : Folio) : Category.{0} W.E := categoryOfElements W.H
217
218 instance (W : Folio) (m : (Fin W.length)op) : LinearOrder (W.H.obj m) :=
219   (W.core.obj m.unop).unop.linearOrder
220
221 instance (W : Folio) (x y : W.E) : Subsingleton (x ==> y) where
222   allEq f g := CategoryOfElements.ext W.H f g (Subsingleton.elim _ _)
223
224 def Folio.originIndex (W : Folio) : Fin W.length := <|0, W.positive|>
225 def Folio.originBase (W : Folio) : (Fin W.length)op := op W.originIndex
226 def Folio.originValue (W : Folio) : W.H.obj W.originBase :=
227   W.originEquiv.symm ()
228 def Folio.originElement (W : Folio) : W.E := <|W.originBase, W.originValue|>
229
230 def Folio.lastIndex (W : Folio) : Fin W.length :=
231   <|W.length - 1, Nat.sub_lt W.positive (by omega)|>
232 def Folio.lastBase (W : Folio) : (Fin W.length)op := op W.lastIndex
233
234 theorem Folio.origin_unique (W : Folio) (x : W.H.obj W.originBase) :
235   x = W.originValue := by
236     apply W.originEquiv.injective
237     exact Subsingleton.elim _ _
238
239 def Folio.baseToOrigin (W : Folio) (m : (Fin W.length)op) :
240   m ==> W.originBase := by
241     letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
242     exact (homOfLE (show W.originIndex <= m.unop by
243       change (0 : Fin W.length) <= m.unop
244       exact bot_le)).op
245
246 def Folio.toOrigin (W : Folio) (x : W.E) : x ==> W.originElement :=
247   CategoryOfElements.homMk x W.originElement (W.baseToOrigin x.1)
248   (W.origin_unique _)
249
250 structure Pag where
251   folio : Folio
252
253 def Pag.H (W : Pag) : (Fin W.folio.length)op ==> Type 0 := W.folio.H
254 def Pag.E (W : Pag) : Type 0 := W.folio.E
255
256 instance (W : Pag) : Category.{0} W.E := categoryOfElements W.H
257
258 instance : Category.{0} Pag where
259   Hom X Y := X.E ==> Y.E
260   id X := Unit X.E
261   comp F G := F then G
262   id_comp _ := rfl
263   comp_id _ := rfl
264   assoc _ _ _ := rfl
265
266 /-- Internally, an atlas is stored by its least finite presentation. The
267 `Folio.F` construction repeats its final page along the rest of the spine, so
268 the stabilization and morphism-tail clauses in the extracted definition are
269 enforced by construction. -/
270
271 def Pag.cell (P : Pag) (m : (Fin P.folio.length)op) (k : P.H.obj m) : P.E := <|m, k|>
272
273 def Pag.cellToOrigin (P : Pag) (m : (Fin P.folio.length)op)
274   (k : P.H.obj m) : P.cell m k ==> P.folio.originElement :=
275   P.folio.toOrigin (P.cell m k)
276
277 def IsPagewiseDisjoint (P : Pag) (G : P.E ==> DomIns) : Prop :=
278   forall (m : (Fin P.folio.length)op) (i j : P.H.obj m), i != j ->
279     forall (x : G.obj (P.cell m i)) (y : G.obj (P.cell m j)),
280       G.map (P.cellToOrigin m i) x != G.map (P.cellToOrigin m j) y
281
282 structure Atl where
283   P : Pag
284   G : P.E ==> DomIns
285   disjoint : IsPagewiseDisjoint P G

```

```

286
287 def Atl.H (X : Atl) : (Fin X.P.folio.length)op ==> Type 0 := X.P.H
288 def Atl.E (X : Atl) : Type 0 := X.P.E
289
290 instance (X : Atl) : Category.{0} X.E := categoryOfElements X.H
291
292 structure AtlHom (X Y : Atl) where
293   P : X.E ==> Y.E
294   A : X.G --> P then Y.G
295
296 def AtlHom.identity (X : Atl) : AtlHom X X where
297   P := Unit X.E
298   A := Id X.G
299
300 def AtlHom.comp {X Y Z : Atl} (f : AtlHom X Y) (g : AtlHom Y Z) : AtlHom X Z where
301   P := f.P then g.P
302   A := f.A >> whiskerLeft f.P g.A
303
304 @[ext]
305 theorem AtlHom.ext {X Y : Atl} (f g : AtlHom X Y)
306   (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
307   cases f
308   cases g
309   simp_all
310
311 /-- Heterogeneous extensionality for natural transformations whose target
312 functors have been identified. This keeps dependent transports localized. -/
313 theorem NatTrans.hex_right {C D : Type*} [Category C] [Category D]
314   {F G H : C ==> D} (alpha : F --> G) (beta : F --> H) (h : G = H)
315   (happ : forall X, HEq (alpha.app X) (beta.app X)) : HEq alpha beta := by
316   cases h
317   apply heq_of_eq
318   ext X
319   exact eq_of_heq (happ X)
320
321 /-- Transporting the codomain of an embedding along an equality of objects is
322 heterogeneously equal to the original embedding. This is the small coherence
323 fact needed whenever stability identifies the image of an origin with the
324 target origin. -/
325 theorem embedding_comp_map_eqToHom_heq {C : Type u} [Category C]
326   (G : CategoryTheory.Functor C DomIns) {a b : C} (h : a = b) {X : DomIns}
327   (e : X --> G.obj a) : HEq (e >> G.map (eqToHom h)) e := by
328   cases h
329   simp
330
331 instance : Category.{0} Atl where
332   Hom := AtlHom
333   id := AtlHom.identity
334   comp := AtlHom.comp
335   id_comp f := by
336     apply AtlHom.ext
337     . rfl
338     . apply heq_of_eq
339     ext x
340     simp [AtlHom.comp, AtlHom.identity]
341   comp_id f := by
342     apply AtlHom.ext
343     . rfl
344     . apply heq_of_eq
345     ext x
346     simp [AtlHom.comp, AtlHom.identity]
347   assoc f g h := by
348     apply AtlHom.ext
349     . rfl
350     . apply heq_of_eq
351     ext x
352     simp [AtlHom.comp]
353
354 def cardinality (A : Atl) : Nat := A.P.folio.length
355
356 def extent (A : Atl) : DomIns := A.G.obj A.P.folio.originElement
357
358 def TerritoryIndex (A : Atl) : Type := A.H.obj A.P.folio.lastBase

```

```

359
360 def territory (A : Atl) (k : TerritoryIndex A) : DomIns :=
361   A.G.obj (A.P.cell A.P.folio.lastBase k)
362
363 def region (A : Atl) (n : TerritoryIndex A) : DomIns := territory A n
364
365 def IsTransposal : MorphismProperty Atl :=
366   fun _ _ F => Function.Injective F.P.obj
367
368 instance : IsTransposal.IsMultiplicative where
369   id_mem _ := Function.injective_id
370   comp_mem _ _ hf hg := hg.comp hf
371
372 abbrev AtlTrap := WideSubcategory IsTransposal
373
374 def AtlTrapInc : AtlTrap ==> Atl := wideSubcategoryInclusion IsTransposal
375
376 def Folio.commonBase (W : Folio) (m n : (Fin W.length)op) :
377   (Fin W.length)op := op (min m.unop n.unop)
378
379 def Folio.toCommonLeft (W : Folio) (m n : (Fin W.length)op) :
380   m --> W.commonBase m n := (homOfLE (min_le_left _ _)).op
381
382 def Folio.toCommonRight (W : Folio) (m n : (Fin W.length)op) :
383   n --> W.commonBase m n := (homOfLE (min_le_right _ _)).op
384
385 def elementLT (X : Atl) (x y : X.E) : Prop :=
386   let m := X.P.folio.commonBase x.1 y.1
387   let l : LinearOrder (X.H.obj m) :=
388     (X.P.folio.core.obj m.unop).unop.linearOrder
389   X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
390     X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2
391
392 /-- The image of a datum in the extent. -/
393 def originImage (X : Atl) (x : X.E) (t : X.G.obj x) : extent X :=
394   X.G.map (X.P.folio.toOrigin x) t
395
396 /-- A datum is covered when its image in the extent comes from a final region. -/
397 def Covered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
398   exists (k : TerritoryIndex X) (l : territory X k),
399     originImage X x t =
400       originImage X (X.P.cell X.P.folio.lastBase k) l
401
402 theorem covered_region (X : Atl) (k : TerritoryIndex X) (l : territory X k) :
403   Covered X (X.P.cell X.P.folio.lastBase k) l :=
404   <|k, l, rfl|>
405
406 /-- The order and coverage conditions from Definition 7.2. -/
407 def IsTraversal : MorphismProperty Atl := fun X Y F =>
408   Function.Injective F.P.obj /\
409     (forall x y, elementLT _ x y -> elementLT _ (F.P.obj x) (F.P.obj y)) /\
410     (forall x (t : X.G.obj x), Covered X x t ->
411       Covered Y (F.P.obj x) (F.A.app x t))
412
413 instance : IsTraversal.IsMultiplicative where
414   id_mem X := by
415     refine <|Function.injective_id, fun _ _ h => h, ?_|>
416     intro x t h
417     simpa [AtlHom.identity] using h
418   comp_mem f g hf hg := by
419     refine <|hg.1.comp hf.1, fun x y h => hg.2.1 _ _ (hf.2.1 _ _ h), ?_|>
420     intro x t h
421     simpa [AtlHom.comp] using hg.2.2 (f.P.obj x) (f.A.app x t) (hf.2.2 x t h)
422
423 abbrev AtlTrav := WideSubcategory IsTraversal
424
425 def AtlTravInc : AtlTrav ==> Atl := wideSubcategoryInclusion IsTraversal
426
427 def AtlTravToAtlTrap : AtlTrav ==> AtlTrap where
428   obj X := WideSubcategory.mk X.obj
429   map f := <|f.1, f.2.1|>
430
431 def IsStableTraversal : MorphismProperty AtlTrav := fun X Y F =>

```

```

432   F.1.P.obj X.obj.P.folio.originElement = Y.obj.P.folio.originElement
433
434   instance : IsStableTraversal.IsMultiplicative where
435     id_mem _ := rfl
436     comp_mem f g hf hg := by
437       change g.1.P.obj (f.1.P.obj _) = _
438       rw [hf, hg]
439
440   abbrev AtlTras := WideSubcategory IsStableTraversal
441
442   def AtlTrasToAtlTrav : AtlTras ==> AtlTrav :=
443     wideSubcategoryInclusion IsStableTraversal
444
445   def AtlTrasInc : AtlTras ==> Atl := AtlTrasToAtlTrav then AtlTravInc
446
447   def IsAtlasMap : ObjectProperty Atl := fun A =>
448     forall v : extent A, Covered A A.P.folio.originElement v
449
450   abbrev AtlMap := IsAtlasMap.FullSubcategory
451
452   def AtlMapInc : AtlMap ==> Atl := ObjectProperty.iota IsAtlasMap
453
454   def IsAtlasMapTrav : ObjectProperty AtlTrav := fun A => IsAtlasMap A.obj
455   abbrev AtlTravMap := IsAtlasMapTrav.FullSubcategory
456
457   def AtlTravMapInc : AtlTravMap ==> AtlTrav := ObjectProperty.iota IsAtlasMapTrav
458
459   /-- The dominion of covered data in a cell. -/
460   def coveredDom (X : Atl) (x : X.E) : DomIns where
461     toDom :=
462       { Carrier := {t : X.G.obj x // Covered X x t}
463         rank :=
464           ({ toFun := Subtype.val
465             inj' := Subtype.val_injective } :
466             Function.Embedding {t : X.G.obj x // Covered X x t} (X.G.obj x)).trans
467             (X.G.obj x).toDom.rank }
468
469   def coveredMap {X : Atl} {x y : X.E} (f : x --> y) :
470     coveredDom X x --> coveredDom X y where
471     toFun t := <|X.G.map f t.1, by
472       rcases t.2 with <|k, l, h|>
473       refine <|k, l, ?_|>
474       have e : f >> X.P.folio.toOrigin y = X.P.folio.toOrigin x :=
475         CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
476       change (X.G.map f >> X.G.map (X.P.folio.toOrigin y)) t.1 = _
477       rw [<- X.G.map_comp, e]
478       exact h|>
479     inj' := fun a b h => Subtype.ext <| (X.G.map f).injective <| congrArg Subtype.val h
480
481   def coveredFunctor (X : Atl) : X.E ==> DomIns where
482     obj := coveredDom X
483     map := coveredMap
484     map_id x := by
485       apply DomIns.hom_ext
486       intro t
487       apply Subtype.ext
488       change (X.G.map (Id x)) t.1 = t.1
489       rw [X.G.map_id]
490       change Function.Embedding.refl _ t.1 = t.1
491       rfl
492     map_comp f g := by
493       apply DomIns.hom_ext
494       intro t
495       apply Subtype.ext
496       change (X.G.map (f >> g)) t.1 =
497         (X.G.map f >> X.G.map g) t.1
498       rw [X.G.map_comp]
499
500   def chartAtlas (X : Atl) : Atl where
501     P := X.P
502     G := coveredFunctor X
503     disjoint := by
504       intro m i j hij x y h

```

```

505     apply X.disjoint m i j hij x.1 y.1
506     exact congrArg Subtype.val h
507
508 theorem chart_all_covered (X : Atl) (x : (chartAtlas X).E)
509   (t : (chartAtlas X).G.obj x) : Covered (chartAtlas X) x t := by
510   rcases t.2 with <|k, l, h|>
511   let l' : territory (chartAtlas X) k := <|l, covered_region X k l|>
512   exact <|k, l', Subtype.ext h|>
513
514 theorem chart_isAtlasMap (X : Atl) : IsAtlasMap (chartAtlas X) :=
515   fun v => chart_all_covered X _ v
516
517 def chartCounit (X : Atl) : chartAtlas X --> X where
518   P := Unit X.E
519   A :=
520     { app := fun _ =>
521       { toFun := Subtype.val
522         inj' := Subtype.val_injective }
523       naturality := by intros; ext; rfl }
524
525 def chartMap {X Y : AtlTrav} (f : X --> Y) :
526   chartAtlas X.obj --> chartAtlas Y.obj where
527   P := f.1.P
528   A :=
529     { app := fun x =>
530       { toFun := fun t => <|f.1.A.app x t.1, f.2.2.2 x t.1 t.2|>
531         inj' := fun a b h => Subtype.ext <| (f.1.A.app x).injective <|
532           congrArg Subtype.val h }
533       naturality := by
534         intro x y g
535         apply DomIns.hom_ext
536         intro t
537         apply Subtype.ext
538         exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t.1 }
539
540 theorem chartMap_isTraversal {X Y : AtlTrav} (f : X --> Y) :
541   IsTraversal (chartMap f) := by
542   refine <|f.2.1, f.2.2.1, ?_|>
543   intro x t _
544   exact chart_all_covered Y.obj _ _
545
546 def Chr : AtlTrav ==> AtlTravMap where
547   obj X := <|WideSubcategory.mk (chartAtlas X.obj), chart_isAtlasMap X.obj|>
548   map f := <|chartMap f, chartMap_isTraversal f|>
549   map_id _ := by
550     apply Subtype.ext
551     change chartMap (Id _) = AtlHom.identity _
552     apply AtlHom.ext
553     . rfl
554     . apply heq_of_eq
555     ext x t
556     rfl
557   map_comp f g := by
558     apply Subtype.ext
559     change chartMap (_ >> _) = AtlHom.comp (chartMap _) (chartMap _)
560     apply AtlHom.ext
561     . rfl
562     . apply heq_of_eq
563     ext x t
564     rfl
565
566 /-- The formal construction realizes the proof sketch by retaining in each
567 cell exactly the data covered by final regions; its counit is the canonical
568 inclusion into the original atlas. -/
569
570 theorem originImage_origin (X : Atl) (v : extent X) :
571   originImage X X.P.folio.originElement v = v := by
572   have e : X.P.folio.toOrigin X.P.folio.originElement =
573     Id X.P.folio.originElement :=
574     CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
575   rw [originImage, e, X.G.map_id]
576   change Function.Embedding.refl _ v = v
577   rfl

```

```

578
579 theorem atlasMap_all_covered {X : Atl} (hX : IsAtlasMap X)
580   (x : X.E) (t : X.G.obj x) : Covered X x t := by
581   rcases hX (originImage X x t) with <|k, l, h|>
582   exact <|k, l, by simp [originImage_origin] using h|>
583
584 theorem chartCounit_isTraversal (X : Atl) : IsTraversal (chartCounit X) := by
585   refine <|Function.injective_id, fun _ _ h => h, ?_|>
586   intro x t _
587   simp [chartCounit] using t.2
588
589 def chartCounitTrav (X : AtlTrav) :
590   (WideSubcategory.mk (chartAtlas X.obj) : AtlTrav) --> X :=
591   <|chartCounit X.obj, chartCounit_isTraversal X.obj|>
592
593 def chartLift {A : AtlTravMap} {X : AtlTrav}
594   (f : AtlTravMapInc.obj A --> X) :
595   A --> Chr.obj X := by
596   refine <|?, ?_|>
597   . exact
598     { P := f.1.P
599       A :=
600         { app := fun x =>
601           { toFun := fun t => <|f.1.A.app x t,
602             f.2.2.2 x t (atlasMap_all_covered A.property x t)|>
603             inj' := by
604               intro a b h
605               apply (f.1.A.app x).injective
606               exact congrArg (fun z => z.1) h }
607             naturality := by
608               intro x y g
609               apply DomIns.hom_ext
610               intro t
611               apply Subtype.ext
612               exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t } }
613     . refine <|f.2.1, f.2.2.1, ?_|>
614       intro x t _
615       exact chart_all_covered X.obj _ _
616
617 def chartHomEquiv (A : AtlTravMap) (X : AtlTrav) :
618   (AtlTravMapInc.obj A --> X) Equiv (A --> Chr.obj X) where
619   toFun := chartLift
620   invFun g := g >> chartCounitTrav X
621   left_inv f := by
622     apply Subtype.ext
623     apply AtlHom.ext
624     . rfl
625     . apply heq_of_eq
626       ext x t
627       rfl
628   right_inv g := by
629     apply Subtype.ext
630     apply AtlHom.ext
631     . rfl
632     . apply heq_of_eq
633       ext x t
634       apply Subtype.ext
635       rfl
636
637 theorem chartHomEquiv_naturality_left {A A' : AtlTravMap} (f : A' --> A)
638   {X : AtlTrav} (g : AtlTravMapInc.obj A --> X) :
639   chartHomEquiv A' X (AtlTravMapInc.map f >> g) =
640   f >> chartHomEquiv A X g := by
641   apply Subtype.ext
642   apply AtlHom.ext
643   . rfl
644   . apply heq_of_eq
645     ext x t
646     apply Subtype.ext
647     rfl
648
649 theorem chartHomEquiv_naturality_right {A : AtlTravMap} {X X' : AtlTrav}
650   (f : AtlTravMapInc.obj A --> X) (g : X --> X') :

```

```

651   chartHomEquiv A X' (f >> g) = chartHomEquiv A X f >> Chr.map g := by
652   apply Subtype.ext
653   apply AtlHom.ext
654   . rfl
655   . apply heq_of_eq
656   ext x t
657   apply Subtype.ext
658   rfl
659
660 def cartographyAdjunction : AtlTravMapInc -| Chr :=
661   Adjunction.mkOfHomEquiv
662   { homEquiv := chartHomEquiv
663     homEquiv_naturality_left_symm := by
664       intro X' X Y f g
665       apply (chartHomEquiv X' Y).injective
666       simpa using (chartHomEquiv_naturality_left f ((chartHomEquiv X Y).symm g)).symm
667     homEquiv_naturality_right := by
668       intro X Y Y' f g
669       exact chartHomEquiv_naturality_right f g }
670
671 theorem cartographyLemma : Nonempty (AtlTravMapInc -| Chr) :=
672   <|cartographyAdjunction|>
673
674 def extentMap {X Y : Atl} (f : X --> Y) : extent X --> extent Y :=
675   f.A.app X.P.folio.originElement >>
676     Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement))
677
678 theorem extentMap_id (X : Atl) : extentMap (Id X) = Id (extent X) := by
679   apply DomIns.hom_ext
680   intro t
681   change (X.G.map (X.P.folio.toOrigin X.P.folio.originElement)) t = t
682   simpa [originImage] using originImage_origin X t
683
684 theorem extentMap_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
685   extentMap (f >> g) = extentMap f >> extentMap g := by
686   apply DomIns.hom_ext
687   intro t
688   let oX := X.P.folio.originElement
689   let y := f.P.obj oX
690   let oY := Y.P.folio.originElement
691   let z := g.P.obj y
692   let z_0 := g.P.obj oY
693   have hn := g.A.naturality (Y.P.folio.toOrigin y)
694   have hz : g.P.map (Y.P.folio.toOrigin y) >> Z.P.folio.toOrigin z_0 =
695     Z.P.folio.toOrigin z :=
696     CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
697   have hn' : Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY =
698     g.A.app y >> Z.G.map (g.P.map (Y.P.folio.toOrigin y)) := hn
699   have hm : g.A.app y >> Z.G.map (Z.P.folio.toOrigin z) =
700     Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY >>
701     Z.G.map (Z.P.folio.toOrigin z_0) := by
702     rw [<- Category.assoc, hn', Category.assoc, <- Z.G.map_comp, hz]
703   exact congrFun (congrArg Function.Embedding.toFun
704     (by simpa only [Category.assoc] using congrArg (fun q => f.A.app oX >> q) hm)) t
705
706 def Ex : Atl ==> DomIns where
707   obj := extent
708   map := extentMap
709   map_id := extentMap_id
710   map_comp := extentMap_comp
711
712 def stableCoalitionMap {X Y : AtlTras} (f : X --> Y) :
713   coveredDom X.obj.obj X.obj.obj.P.folio.originElement -->
714   coveredDom Y.obj.obj Y.obj.obj.P.folio.originElement where
715   toFun t := by
716     have hc := f.1.2.2.2 X.obj.obj.P.folio.originElement t.1 t.2
717     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
718     <|f.1.1.A.app X.obj.obj.P.folio.originElement t.1, hc|>
719   inj' := fun a b h => by
720     apply Subtype.ext
721     apply (f.1.1.A.app X.obj.obj.P.folio.originElement).injective
722     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
723     exact congrArg Subtype.val h

```

```

724
725 theorem stableCoalitionMap_val {X Y : At1Tras} (f : X --> Y)
726   (t : coveredDom X.obj.obj X.obj.obj.P.folio.originElement) :
727   (stableCoalitionMap f t).1 = extentMap f.1.1 t.1 := by
728   change Y.obj.obj.G.map (eqToHom f.2)
729   (f.1.1.A.app X.obj.obj.P.folio.originElement t.1) =
730   Y.obj.obj.G.map (Y.obj.obj.P.folio.toOrigin
731     (f.1.1.P.obj X.obj.obj.P.folio.originElement))
732   (f.1.1.A.app X.obj.obj.P.folio.originElement t.1)
733   have hmor : eqToHom f.2 = Y.obj.obj.P.folio.toOrigin
734     (f.1.1.P.obj X.obj.obj.P.folio.originElement) := by
735     apply CategoryOfElements.ext
736     apply Subsingleton.elim
737   rw [hmr]
738
739 /-- `Coa` is written directly on the covered origins. This is definitionally
740 the object part of `Ex o Chr`; stability makes its action on arrows reduce to
741 the component at the origin. -/
742 def Coa : At1Tras ==> DomIns where
743   obj X := coveredDom X.obj.obj X.obj.obj.P.folio.originElement
744   map := stableCoalitionMap
745   map_id X := by
746     apply DomIns.hom_ext
747     intro t
748     apply Subtype.ext
749     rw [stableCoalitionMap_val]
750     exact congrFun (congrArg Function.Embedding.toFun (extentMap_id X.obj.obj)) t.1
751   map_comp f g := by
752     apply DomIns.hom_ext
753     intro t
754     apply Subtype.ext
755     rw [stableCoalitionMap_val]
756     change extentMap (f.1.1 >> g.1.1) t.1 =
757       (stableCoalitionMap g (stableCoalitionMap f t)).1
758     rw [stableCoalitionMap_val, stableCoalitionMap_val]
759     exact congrFun (congrArg Function.Embedding.toFun
760       (extentMap_comp f.1.1 g.1.1)) t.1
761
762 def coalition (X : At1) : DomIns := (Coa.obj
763   (WideSubcategory.mk (WideSubcategory.mk X) : At1Tras))
764
765 /-- The initial dominion. -/
766 def emptyDominion : DomIns where
767   toDom :=
768     { Carrier := Empty
769       rank :=
770         { toFun := fun x => nomatch x
771           inj' := fun x => nomatch x } }
772
773 def singletonChain : Chain where
774   Obj := Unit
775   linearOrder := inferInstance
776   wellFoundedLT := inferInstance
777   orderType_lt := by
778     have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
779       Ordinal.omega0 ^ Ordinal.omega0 :=
780       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
781     Ordinal.one_lt_omega0
782     have homega : Ordinal.omega0 < Ordinal.omega0 ^ Ordinal.omega0 := by
783       simp only [Ordinal.opow_one] using hpow
784       simp using lt_trans Ordinal.one_lt_omega0 homega
785
786 def singletonObjEquiv : singletonChain.Obj Equiv Unit := Equiv.refl _
787
788 def onePageFolio : Folio where
789   length := 1
790   positive := by omega
791   core := (Functor.const (Fin 1)).obj (op singletonChain)
792   originEquiv := singletonObjEquiv
793
794 def onePagePag : Pag := <|onePageFolio|>
795
796 theorem onePage_cell_unique (x : onePagePag.E) :

```



```

797     x = onePageFolio.originElement := by
798   let hbase : x.1 = onePageFolio.originBase := by
799     apply unop_injective
800     apply Fin.eq_zero
801   apply Functor.Elements.ext x onePageFolio.originElement hbase
802   exact onePageFolio.origin_unique _
803
804   def dominionAtlas (X : DomIns) : Atl where
805     P := onePagePag
806     G := (Functor.const onePagePag.E).obj X
807     disjoint := by
808       intro m i j hij
809       exfalse
810       apply hij
811     apply onePageFolio.originEquiv.injective
812     exact Subsingleton.elim _ _
813
814   theorem dominionAtlas_isMap (X : DomIns) : IsAtlasMap (dominionAtlas X) := by
815     intro v
816     let k : TerritoryIndex (dominionAtlas X) :=
817       onePageFolio.originValue
818     refine <|k, v, ?_|>
819     rw [originImage_origin]
820     have hc : (dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k =
821       (dominionAtlas X).P.folio.originElement := onePage_cell_unique _
822     rw [hc, originImage_origin]
823
824   def dominionMap {X Y : DomIns} (f : X --> Y) :
825     dominionAtlas X --> dominionAtlas Y where
826     P := Unit onePagePag.E
827     A :=
828       { app := fun _ => f
829         naturality := by intros; apply DomIns.hom_ext; intro; rfl }
830
831   theorem dominionMap_isTraversal {X Y : DomIns} (f : X --> Y) :
832     IsTraversal (dominionMap f) := by
833       refine <|Function.injective_id, fun _ _ h => h, ?_|>
834       intro x t
835       exact atlasMap_all_covered (dominionAtlas_isMap Y) _ _
836
837   theorem dominionMap_isStable {X Y : DomIns} (f : X --> Y) :
838     IsStableTraversal
839       (X := WideSubcategory.mk (dominionAtlas X))
840       (Y := WideSubcategory.mk (dominionAtlas Y))
841       <|dominionMap f, dominionMap_isTraversal f|> := rfl
842
843   def DomInc : DomIns ==> AtlTras where
844     obj X := WideSubcategory.mk (WideSubcategory.mk (dominionAtlas X))
845     map f := <|<|dominionMap f, dominionMap_isTraversal f|>,
846       dominionMap_isStable f|>
847     map_id _ := by
848       apply Subtype.ext
849       apply Subtype.ext
850       apply AtlHom.ext
851       . rfl
852       . apply heq_of_eq
853         ext
854         rfl
855     map_comp f g := by
856       apply Subtype.ext
857       apply Subtype.ext
858       apply AtlHom.ext
859       . rfl
860       . apply heq_of_eq
861         ext x t
862         change (f >> g) t = (f >> g) t
863       rfl
864
865   def onePageToAtlasP (Y : Atl) : onePagePag.E ==> Y.E :=
866     (Functor.const onePagePag.E).obj Y.P.folio.originElement
867
868   theorem onePageToAtlasP_injective (Y : Atl) :
869     Function.Injective (onePageToAtlasP Y).obj := by

```

```

870   intro x y _
871   exact onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
872
873 theorem onePage_elementLT_false (X : DomIns) (x y : (dominionAtlas X).E) :
874   not elementLT (dominionAtlas X) x y := by
875   intro h
876   have hxy : x = y := onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
877   subst y
878   let m := (dominionAtlas X).P.folio.commonBase x.1 x.1
879   let l : LinearOrder ((dominionAtlas X).H.obj m) :=
880     ((dominionAtlas X).P.folio.core.obj m.unop).unop.linearOrder
881   exact lt_irrefl _ h
882
883 /-- A stable traversal from a one-page atlas determines an embedding into
884 the covered part of the target extent. -/
885 def domIncToCoa {X : DomIns} {Y : AtlTras} (f : DomInc.obj X --> Y) :
886   X --> Coa.obj Y where
887   toFun x := by
888     have hc := f.1.2.2.2 onePageFolio.originElement x
889     (atlasMap_all_covered (dominionAtlas_isMap X) _ x)
890     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
891     <|f.1.1.A.app onePageFolio.originElement x, hc|>
892   inj' := fun a b h => by
893     apply (f.1.1.A.app onePageFolio.originElement).injective
894     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
895     exact congrArg Subtype.val h
896
897 /-- Conversely, an embedding into the coalition supplies the unique stable
898 traversal from the corresponding one-page atlas. -/
899 def coaToDomInc {X : DomIns} {Y : AtlTras} (f : X --> Coa.obj Y) :
900   DomInc.obj X --> Y := by
901   let p : (dominionAtlas X).E ==> Y.obj.obj.P.E := onePageToAtlasP Y.obj.obj
902   let a : (dominionAtlas X).G --> p then Y.obj.obj.G :=
903     { app := fun _ =>
904       { toFun := fun x => (f x).1
905         inj' := fun x y h => f.injective (Subtype.ext h) }
906     naturality := by
907       intro x y g
908       apply DomIns.hom_ext
909       intro t
910       have hxy : x = y := onePage_cell_unique x |>.trans
911         (onePage_cell_unique y).symm
912       subst y
913       have hg : g = Id x := by
914         apply CategoryOfElements.ext
915         apply Subsingleton.elim
916       subst g
917       simp [p] }
918   let h : dominionAtlas X --> Y.obj.obj := <|p, a|>
919   have htrav : IsTraversal h := by
920     refine <|onePageToAtlasP_injective Y.obj.obj, ?, ?_|>
921     . intro x y hxy
922     exact (onePage_elementLT_false X x y hxy).elim
923     . intro x t _
924     change Covered Y.obj.obj Y.obj.obj.P.folio.originElement (f t).1
925     exact (f t).2
926   exact <|<|h, htrav|>, rfl|>
927
928 def coaDomIncIso (X : DomIns) : Coa.obj (DomInc.obj X) Iso X where
929   hom :=
930     { toFun := fun x => x.1
931       inj' := fun x y h => Subtype.ext h }
932   inv :=
933     { toFun := fun x => <|x, atlasMap_all_covered
934       (dominionAtlas_isMap X) onePageFolio.originElement x|>
935       inj' := fun x y h => congrArg Subtype.val h }
936   hom_inv_id := by
937     apply DomIns.hom_ext
938     intro x
939     apply Subtype.ext
940     rfl
941   inv_hom_id := by
942     apply DomIns.hom_ext

```

```

943     intro x
944     rfl
945
946     /-- The hom-set equivalence expressing that one-page insertion is left
947     adjoint to coalization. -/
948   def domIncCoaHomEquiv (X : DomIns) (Y : AtlTras) :
949     (DomInc.obj X --> Y) Equiv (X --> Coa.obj Y) where
950     toFun := domIncToCoa
951     invFun := coaToDomInc
952     left_inv := by
953       intro f
954       apply Subtype.ext
955       apply Subtype.ext
956       have hP : (coaToDomInc (domIncToCoa f)).1.1.P = f.1.1.P := by
957         exact CategoryTheory.Functor.ext
958           (fun x => f.2.symm.trans
959             (congrArg f.1.1.P.obj (onePage_cell_unique x).symm))
960           (fun _ _ => by
961             apply CategoryOfElements.ext
962             apply Subsingleton.elim)
963       apply AtlHom.ext _ _ hP
964       apply NatTrans.hext_right _ _
965       (congrArg (fun Q => Q then Y.obj.obj.G) hP)
966     intro x
967     dsimp [coaToDomInc, domIncToCoa, onePageToAtlasP]
968     have hx : x = onePageFolio.originElement := onePage_cell_unique x
969     subst x
970     let e := f.1.1.A.app onePageFolio.originElement
971     have hrec :
972       { toFun := fun x =>
973         (coveredMap (X := Y.obj.obj) (eqToHom f.2)
974           <|e x, by
975             exact f.1.2.2.2 onePageFolio.originElement x
976             (atlasMap_all_covered (dominionAtlas_isMap X) _ x)|>).1
977         inj' := by
978           intro a b hab
979           apply e.injective
980           apply (Y.obj.obj.G.map (eqToHom f.2)).injective
981           exact hab } = e >> Y.obj.obj.G.map (eqToHom f.2) := by
982       apply DomIns.hom_ext
983       intro t
984       rfl
985       exact (heq_of_eq hrec).trans
986         (embedding_comp_map_eqToHom_heq Y.obj.obj.G f.2 e)
987     right_inv := by
988       intro f
989       apply DomIns.hom_ext
990       intro x
991       apply Subtype.ext
992       change Y.obj.obj.G.map (Id Y.obj.obj.P.folio.originElement) (f x).1 = (f x).1
993       exact congrFun (congrArg Function.Embedding.toFun
994         (Y.obj.obj.G.map_id Y.obj.obj.P.folio.originElement)) (f x).1
995
996   theorem domIncCoaHomEquiv_naturality_left {X X' : DomIns} (f : X' --> X)
997     {Y : AtlTras} (g : DomInc.obj X --> Y) :
998     domIncCoaHomEquiv X' Y (DomInc.map f >> g) =
999     f >> domIncCoaHomEquiv X Y g := by
1000     apply DomIns.hom_ext
1001     intro x
1002     apply Subtype.ext
1003     rfl
1004
1005   theorem domIncCoaHomEquiv_naturality_right {X : DomIns} {Y Y' : AtlTras}
1006     (f : DomInc.obj X --> Y) (g : Y --> Y') :
1007     domIncCoaHomEquiv X Y' (f >> g) =
1008     domIncCoaHomEquiv X Y f >> Coa.map g := by
1009     apply DomIns.hom_ext
1010     intro x
1011     apply Subtype.ext
1012     let t : Coa.obj (DomInc.obj X) :=
1013       <|x, atlasMap_all_covered
1014         (dominionAtlas_isMap X) onePageFolio.originElement x|>
1015     change (Coa.map (f >> g) t).1 = (Coa.map g (Coa.map f t)).1

```

```

1016 exact congrArg Subtype.val <| congrFun
1017   (congrArg Function.Embedding.toFun (Coa.map_comp f g)) t
1018
1019 /-- The unconditional adjunction promised by the Domanial Inclusion
1020 construction. -/
1021 def dominionAdjunction : DomInc -| Coa :=
1022   Adjunction.mkOfHomEquiv
1023     { homEquiv := domIncCoaHomEquiv
1024       homEquiv_naturality_left_symm := by
1025         intro X' X Y f g
1026         apply (domIncCoaHomEquiv X' Y).injective
1027         simpa using
1028           (domIncCoaHomEquiv_naturality_left f
1029             ((domIncCoaHomEquiv X Y).symm g)).symm
1030       homEquiv_naturality_right := by
1031         intro X Y Y' f g
1032         exact domIncCoaHomEquiv_naturality_right f g }
1033
1034 theorem dominionLemma : Nonempty (DomInc -| Coa) :=
1035   <|dominionAdjunction|>
1036
1037 abbrev DaTra := Atlop ==> Type
1038
1039 def Yo : Atl ==> DaTra := yoneda
1040
1041 /-- A concrete certificate of the statement that `DaTra` is the displayed
1042 presheaf category. -/
1043 def datraToposPresentation : DaTra EquivCat (Atlop ==> Type) :=
1044   CategoryTheory.Equivalence.refl
1045
1046 structure Navigation (D : DaTra) where
1047   A : Atl
1048   hom : Yo.obj A --> D
1049   mono : Mono hom
1050
1051 attribute [instance] Navigation.mono
1052
1053 structure Expedition (D : DaTra) extends Navigation D where
1054   atlasMap : IsAtlasMap A
1055
1056 /-- Adjoin the empty presheaf to the image of Yoneda. Its new source object
1057 represents exactly the empty alternative in a DaTra restriction. -/
1058 def PartialYo : WithInitial Atl ==> DaTra :=
1059   WithInitial.liftToInitial Yo (initialIsInitial)
1060
1061 def liftWide (W : MorphismProperty Atl) :
1062   MorphismProperty (WithInitial Atl) := fun X Y f =>
1063     match X, Y with
1064     | .star, _ => True
1065     | .of _, .of _ => W (WithInitial.down f)
1066
1067 instance (W : MorphismProperty Atl) [W.IsMultiplicative] :
1068   (liftWide W).IsMultiplicative where
1069   id_mem X := by
1070     cases X with
1071     | star => trivial
1072     | of X => exact W.id_mem X
1073   comp_mem {X Y Z} f g hf hg := by
1074     cases X with
1075     | star => trivial
1076     | of X =>
1077       cases Y with
1078       | star => exact nomatch f
1079       | of Y =>
1080         cases Z with
1081         | star => exact nomatch g
1082         | of Z => exact W.comp_mem _ _ hf hg
1083
1084 /-- Existential relative representability. Unlike the representation-
1085 independent variant in Mathlib, this is the exact condition in the text:
1086 there is one representing pullback whose atlas arrow lies in `W`. -/
1087 def HasRestrictedPullbacks (W : MorphismProperty (WithInitial Atl)) :
1088   MorphismProperty DaTra := fun X Y F =>

```

```

1089 forall {{A : WithInitial Atl}} (nav : PartialYo.obj A --> Y),
1090   exists (B : WithInitial Atl) (f : B --> A) (nav' : PartialYo.obj B --> X),
1091     IsPullback nav' (PartialYo.map f) F nav /\ W f
1092
1093 instance (W : MorphismProperty (WithInitial Atl)) [W.IsMultiplicative] :
1094   (HasRestrictedPullbacks W).IsMultiplicative where
1095   id_mem X := by
1096     intro A nav
1097     exact <|A, Id A, nav, by simp using IsPullback.of_id_snd, W.id_mem A|>
1098   comp_mem f g hf hg := by
1099     intro A nav
1100     obtain <|B, q, navB, hBg, hq|> := hg nav
1101     obtain <|C, p, navC, hCf, hp|> := hf navB
1102     exact <|C, p >> q, navC,
1103       by simp using IsPullback.paste_vert hCf hBg,
1104       W.comp_mem p q hp hq|>
1105
1106 abbrev DaTraRestriction (W : MorphismProperty Atl) [W.IsMultiplicative] :=
1107   WideSubcategory (HasRestrictedPullbacks (liftWide W))
1108
1109 abbrev DaTrap := DaTraRestriction IsTransposal
1110 abbrev DaTrav := DaTraRestriction IsTraversal
1111
1112 def IsStableAtl : MorphismProperty Atl := fun X Y f =>
1113   exists (hf : IsTraversal f), IsStableTraversal
1114     (X := WideSubcategory.mk X) (Y := WideSubcategory.mk Y) <|f, hf|>
1115
1116 instance : IsStableAtl.IsMultiplicative where
1117   id_mem X := <|IsTraversal.id_mem X, IsStableTraversal.id_mem _|>
1118   comp_mem f g hf hg :=
1119     <|IsTraversal.comp_mem f g hf.1 hg.1,
1120       IsStableTraversal.comp_mem <|f, hf.1|> <|g, hg.1|> hf.2 hg.2|>
1121
1122 abbrev DaTras := DaTraRestriction IsStableAtl
1123
1124 def IsDaTraMap : ObjectProperty DaTra := fun D =>
1125   forall nav : Navigation D, IsAtlasMap nav.A
1126
1127 abbrev DaTraMap := IsDaTraMap.FullSubcategory
1128
1129 def AtlI : Atl := dominionAtlas emptyDominion
1130
1131 theorem AtlI_eq_domInc_zero :
1132   AtlI = (DomInc.obj emptyDominion).obj.obj := rfl
1133
1134 def Folio.pageValue (W : Folio) (m : Fin W.length) :
1135   (W.core.obj m).unop.Obj := by
1136   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
1137   let q : (W.core.obj m).unop --> (W.core.obj W.originIndex).unop :=
1138     (W.core.map (homOfLE (Fin.zero_le m))).unop
1139   exact Classical.choose (q.point_surjective W.originValue)
1140
1141 theorem Folio.map_unop_comp_apply (W : Folio) {i j k : Fin W.length}
1142   (hik : i --> k) (hij : i --> j) (hjk : j --> k)
1143   (z : (W.core.obj k).unop.Obj) :
1144   (W.core.map hik).unop z =
1145     (W.core.map hij).unop ((W.core.map hjk).unop z) := by
1146   have e : hik = hij >> hjk := Subsingleton.elim _ _
1147   subst hik
1148   have h := congrArg Quiver.Hom.unop (W.core.map_comp hij hjk)
1149   exact congrFun (congrArg ConHom.toFun h) z
1150
1151 def bouquetLength (X Y : Folio) : Nat := max X.length Y.length + 1
1152
1153 def bouquetDepth (X Y : Folio) : Nat := max X.length Y.length
1154
1155 theorem bouquetLength_pos (X Y : Folio) : 0 < bouquetLength X Y := by
1156   simp [bouquetLength]
1157
1158 instance (X Y : Folio) : NeZero (bouquetLength X Y) :=
1159   <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1160
1161 def bouquetChain (X Y : Folio) : Fin (bouquetLength X Y) -> Chain :=

```

```

1162 Fin.cases singletonChain (fun m : Fin (bouquetDepth X Y) =>
1163   Chain.sum (X.core.obj (X.paddedIndex m.1)).unop
1164   (Y.core.obj (Y.paddedIndex m.1)).unop)
1165
1166 def bouquetValue (X Y : Folio) (m : Fin (bouquetLength X Y)) :
1167   (bouquetChain X Y m).Obj :=
1168   Fin.cases () (fun n => toLex (Sum.inl (X.pageValue (X.paddedIndex n.1)))) m
1169
1170 def bouquetConMap (X Y : Folio) {i j : Fin (bouquetLength X Y)} (h : i <= j) :
1171   bouquetChain X Y j --> bouquetChain X Y i := by
1172   revert j
1173   refine Fin.cases ?_ (fun i => ?_) i
1174   . rename_i j
1175   intro h
1176   refine
1177     { toFun := fun _ => ()
1178       monotone := fun _ _ => le_refl
1179       point_surjective := fun z => by
1180         refine <|bouquetValue X Y j, ?_|>
1181         change (()) : Unit = z
1182         exact Subsingleton.elim _ _ }
1183   . rename_i j
1184   intro h
1185   revert h
1186   refine Fin.cases (by
1187     intro h
1188     change i.1 + 1 <= 0 at h
1189     omega) (fun j => ?_) j
1190   intro h
1191   have hij : i <= j := Fin.succ_le_succ_iff.mp h
1192   let hx : X.paddedIndex i.1 <= X.paddedIndex j.1 := X.paddedIndex_mono hij
1193   let hy : Y.paddedIndex i.1 <= Y.paddedIndex j.1 := Y.paddedIndex_mono hij
1194   exact ConHom.sum (X.core.map (homOfLE hx)).unop
1195     (Y.core.map (homOfLE hy)).unop
1196
1197 def bouquetFolio (X Y : Folio) : Folio where
1198   length := bouquetLength X Y
1199   positive := bouquetLength_pos X Y
1200   core :=
1201     { obj := fun m => op (bouquetChain X Y m)
1202       map := fun f => (bouquetConMap X Y (leOfHom f)).op
1203       map_id := by
1204         intro m
1205         refine Fin.cases ?_ (fun m => ?_) m
1206         . apply Quiver.Hom.unop_inj
1207         apply ConHom.ext
1208         intro z
1209         change (()) : Unit = ()
1210         rfl
1211       apply Quiver.Hom.unop_inj
1212       apply ConHom.ext
1213       intro w
1214       generalize hz : ofLex w = z
1215       rcases z with z | z
1216       . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simp using hz)
1217         subst w
1218         simp [bouquetConMap, ConHom.sum]
1219         change toLex (Sum.inl z) = toLex (Sum.inl z)
1220         rfl
1221       . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simp using hz)
1222         subst w
1223         simp [bouquetConMap, ConHom.sum]
1224         change toLex (Sum.inr z) = toLex (Sum.inr z)
1225         rfl
1226       map_comp := by
1227         intro i j k f g
1228         letI : NeZero (bouquetLength X Y) := <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1229         revert j k
1230         refine Fin.cases ?_ (fun i => ?_) i
1231         . rename_i jTarget kTarget
1232         intro f g
1233         apply Quiver.Hom.unop_inj
1234         apply ConHom.ext

```

```

1235     intro z
1236     change ((() : Unit) = ())
1237     rfl
1238     . rename_i iOrig jTarget kTarget
1239     intro f g
1240     have hj : jTarget != 0 := by
1241       intro e
1242       have hf := leOfHom f
1243       rw [e] at hf
1244       change i.1 + 1 <= 0 at hf
1245       omega
1246     obtain <|j, rfl|> := Fin.eq_succ_of_ne_zero hj
1247     have hk : kTarget != 0 := by
1248       intro e
1249       have hg := leOfHom g
1250       rw [e] at hg
1251       change j.1 + 1 <= 0 at hg
1252       omega
1253     obtain <|k, rfl|> := Fin.eq_succ_of_ne_zero hk
1254     apply Quiver.Hom.unop_inj
1255     apply ConHom.ext
1256     intro w
1257     generalize hz : ofLex w = z
1258     rcases z with z | z
1259     . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simp using hz)
1260       subst w
1261       simp only [bouquetConMap, ConHom.sum] using congrArg
1262       (fun q => toLex (Sum.inl q)) (X.map_unop_comp_apply _ _ _ z)
1263     . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simp using hz)
1264       subst w
1265       simp only [bouquetConMap, ConHom.sum] using congrArg
1266       (fun q => toLex (Sum.inr q)) (Y.map_unop_comp_apply _ _ _ z) }
1267   originEquiv := by
1268     exact singletonObjEquiv
1269
1270   def domSum (X Y : DomIns) : DomIns where
1271     toDom :=
1272       { Carrier := Sum X Y
1273       rank :=
1274         { toFun := fun z => match z with
1275           | .inl x => 2 * X.toDom.rank x
1276           | .inr y => 2 * Y.toDom.rank y + 1
1277         inj' := by
1278           intro a b h
1279           rcases a with a | a <;> rcases b with b | b
1280           . simp only at h
1281           exact congrArg Sum.inl (X.toDom.rank.injective (by omega))
1282           . simp only at h
1283           omega
1284           . simp only at h
1285           omega
1286           . simp only at h
1287           exact congrArg Sum.inr (Y.toDom.rank.injective (by omega)) } }
1288
1289   def domSum.inl (X Y : DomIns) : X --> domSum X Y where
1290     toFun := Sum.inl
1291     inj' := Sum.inl_injective
1292
1293   def domSum.inr (X Y : DomIns) : Y --> domSum X Y where
1294     toFun := Sum.inr
1295     inj' := Sum.inr_injective
1296
1297   def bouquetPag (X Y : At1) : Pag := <|bouquetFolio X.P.folio Y.P.folio|>
1298
1299   def imageDom {A R : DomIns} (f : A --> R) : DomIns where
1300     toDom :=
1301       { Carrier := Set.range f
1302       rank :=
1303         { toFun := Subtype.val
1304         inj' := Subtype.val_injective } : Set.range f --> R).trans R.toDom.rank }
1305
1306   def imageDom.inclusion {A R : DomIns} (f : A --> R) : imageDom f --> R where
1307     toFun := Subtype.val

```

```

1308   inj' := Subtype.val_injective
1309
1310   def bouquetRawDomIndex (X Y : Atl) :
1311     (m : Fin (bouquetLength X.P.folio Y.P.folio)) ->
1312     (bouquetFolio X.P.folio Y.P.folio).H.obj (op m) -> DomIns :=
1313     Fin.cases (fun _ => domSum (extent X) (extent Y)) (fun n v =>
1314       match ofLex v with
1315       | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
1316       | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
1317
1318   def bouquetRootIndex (X Y : Atl) :
1319     forall (m : Fin (bouquetLength X.P.folio Y.P.folio))
1320       (v : (bouquetFolio X.P.folio Y.P.folio).H.obj (op m)),
1321       bouquetRawDomIndex X Y m v --> domSum (extent X) (extent Y) := by
1322     intro m
1323     induction m using Fin.cases with
1324     | zero => intro v; exact Id _
1325     | succ n =>
1326       intro v
1327       generalize hv : ofLex v = s
1328       rcases s with k | k
1329       . have ev : v = toLex (Sum.inl k) :=
1330         ofLex.injective (by simp using hv)
1331       subst v
1332       exact X.G.map (X.P.cellToOrigin
1333         (op (X.P.folio.paddedIndex n.1)) k) >>
1334         domSum.inl (extent X) (extent Y)
1335       . have ev : v = toLex (Sum.inr k) :=
1336         ofLex.injective (by simp using hv)
1337       subst v
1338       exact Y.G.map (Y.P.cellToOrigin
1339         (op (Y.P.folio.paddedIndex n.1)) k) >>
1340         domSum.inr (extent X) (extent Y)
1341
1342   def bouquetDom (X Y : Atl) (x : (bouquetPag X Y).E) : DomIns :=
1343     imageDom (bouquetRootIndex X Y x.1.unop x.2)
1344
1345   theorem bouquetRange_mono (X Y : Atl) {x y : (bouquetPag X Y).E}
1346     (f : x --> y) :
1347     Set.range (bouquetRootIndex X Y x.1.unop x.2) subset
1348     Set.range (bouquetRootIndex X Y y.1.unop y.2) := by
1349     intro z hz
1350     rcases x with <|<|mx|>, vx|>
1351     rcases y with <|<|my|>, vy|>
1352     rcases hz with <|a, ha|>
1353     rw [<- ha]
1354     induction mx using Fin.cases with
1355     | zero =>
1356       induction my using Fin.cases with
1357       | zero => exact <|a, rfl|>
1358       | succ p =>
1359         have hf : p.succ -->
1360           (<|0, bouquetLength_pos X.P.folio Y.P.folio|> :
1361             Fin (bouquetLength X.P.folio Y.P.folio)) :=
1362           Quiver.Hom.unop f.val
1363         have h := leOfHom hf
1364         change p.1 + 1 <= 0 at h
1365         omega
1366     | succ n =>
1367       induction my using Fin.cases with
1368       | zero => exact <|bouquetRootIndex X Y n.succ vx a, rfl|>
1369       | succ p =>
1370         have hf : p.succ --> n.succ := Quiver.Hom.unop f.val
1371         have hpn : p <= n := Fin.succ_le_succ_iff.mp (leOfHom hf)
1372         let hxp : X.P.folio.paddedIndex p.1 <= X.P.folio.paddedIndex n.1 :=
1373           X.P.folio.paddedIndex_mono hpn
1374         let hyp : Y.P.folio.paddedIndex p.1 <= Y.P.folio.paddedIndex n.1 :=
1375           Y.P.folio.paddedIndex_mono hpn
1376         generalize hvx : ofLex vx = s
1377         rcases s with k | k
1378         . have evx : vx = toLex (Sum.inl k) :=
1379           ofLex.injective (by simp using hvx)
1380         subst vx

```



```

1381 let k' := X.P.H.map (homOfLE hxp).op k
1382 have ef : f.val = (homOfLE (show p.succ <= n.succ from
1383   Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
1384 have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inl k)) = vy :=
1385   f.property
1386 have hvy : ofLex vy = Sum.inl k' := by
1387   rw [<- hfv, ef]
1388   change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inl k))))) = _
1389   simp [k']
1390   congr 2
1391 have evy : vy = toLex (Sum.inl k') :=
1392   ofLex.injective (by simp using hvy)
1393 subst vy
1394 let q := CategoryOfElements.homMk
1395   (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
1396   (X.P.cell (op (X.P.folio.paddedIndex p.1)) k')
1397   (homOfLE hxp).op rfl
1398 refine <|X.G.map q a, ?_|>
1399 change Sum.inl (X.G.map (X.P.cellToOrigin _ k') (X.G.map q a)) =
1400   Sum.inl (X.G.map (X.P.cellToOrigin _ k) a)
1401 apply congrArg Sum.inl
1402 have hc : q >> X.P.cellToOrigin _ k' = X.P.cellToOrigin _ k :=
1403   CategoryOfElements.ext X.P.H _ _ (Subsingleton.elim _ _)
1404 have hm := X.G.map_comp q (X.P.cellToOrigin _ k')
1405 rw [hc] at hm
1406 exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
1407 have evx : vx = toLex (Sum.inr k) :=
1408   ofLex.injective (by simp using hvx)
1409 subst vx
1410 let k' := Y.P.H.map (homOfLE hyp).op k
1411 have ef : f.val = (homOfLE (show p.succ <= n.succ from
1412   Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
1413 have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inr k)) = vy :=
1414   f.property
1415 have hvy : ofLex vy = Sum.inr k' := by
1416   rw [<- hfv, ef]
1417   change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inr k))))) = _
1418   simp [k']
1419   congr 2
1420 have evy : vy = toLex (Sum.inr k') :=
1421   ofLex.injective (by simp using hvy)
1422 subst vy
1423 let q := CategoryOfElements.homMk
1424   (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)
1425   (Y.P.cell (op (Y.P.folio.paddedIndex p.1)) k')
1426   (homOfLE hyp).op rfl
1427 refine <|Y.G.map q a, ?_|>
1428 change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k') (Y.G.map q a)) =
1429   Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a)
1430 apply congrArg Sum.inr
1431 have hc : q >> Y.P.cellToOrigin _ k' = Y.P.cellToOrigin _ k :=
1432   CategoryOfElements.ext Y.P.H _ _ (Subsingleton.elim _ _)
1433 have hm := Y.G.map_comp q (Y.P.cellToOrigin _ k')
1434 rw [hc] at hm
1435 exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
1436
1437 def imageDom.map {A B R : DomIns} (f : A --> R) (g : B --> R)
1438   (h : Set.range f subset Set.range g) : imageDom f --> imageDom g where
1439   toFun z := <|z.1, h z.2|>
1440   inj' := by
1441     intro a b e
1442     apply Subtype.ext
1443     exact congrArg (fun q => q.1) e
1444
1445 @[simp]
1446 theorem imageDom.map_val {A B R : DomIns} (f : A --> R) (g : B --> R)
1447   (h : Set.range f subset Set.range g) (z : imageDom f) :
1448   (imageDom.map f g h z).1 = z.1 := rfl
1449
1450 def bouquetImageMap (X Y : At1) {x y : (bouquetPag X Y).E} (f : x --> y) :
1451   bouquetDom X Y x --> bouquetDom X Y y :=
1452   imageDom.map _ _ (bouquetRange_mono X Y f)
1453

```

```

1454 def bouquetFunctor (X Y : Atl) : (bouquetPag X Y).E ==> DomIns where
1455   obj := bouquetDom X Y
1456   map := bouquetImageMap X Y
1457   map_id := by
1458     intro x
1459     apply DomIns.hom_ext
1460     intro z
1461     apply Subtype.ext
1462     change z.1 = z.1
1463     rfl
1464   map_comp := by
1465     intro x y z f g
1466     apply DomIns.hom_ext
1467     intro w
1468     apply Subtype.ext
1469     change w.1 = w.1
1470     rfl
1471
1472 def bouquetAtlas (X Y : Atl) : Atl where
1473   P := bouquetPag X Y
1474   G := bouquetFunctor X Y
1475   disjoint := by
1476     intro m i j hij x y hxy
1477     rcases m with <|m|>
1478     induction m using Fin.cases with
1479     | zero =>
1480       exact hij ((bouquetFolio X.P.folio Y.P.folio).originEquiv.injective
1481         (Subsingleton.elim _ _))
1482     | succ n =>
1483       rcases x.2 with <|a, ha|>
1484       rcases y.2 with <|b, hb|>
1485       have huv : x.1 = y.1 := by
1486         have e := congrArg (fun q => q.1) hxy
1487         change x.1 = y.1 at e
1488         exact e
1489       have hroot : bouquetRootIndex X Y n.succ i a =
1490         bouquetRootIndex X Y n.succ j b := ha.trans (huv.trans hb.symm)
1491       generalize hi : ofLex i = si
1492       generalize hj : ofLex j = sj
1493       rcases si with k | k <; rcases sj with l | l
1494       . have ei : i = toLex (Sum.inl k) :=
1495         ofLex.injective (by simp using hi)
1496       have ej : j = toLex (Sum.inl l) :=
1497         ofLex.injective (by simp using hj)
1498       subst i
1499       subst j
1500       have hkl : k != l := by
1501         intro e
1502         subst l
1503         exact hij rfl
1504       change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
1505         Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
1506       exact X.disjoint _ k l hkl a b (Sum.inl.inj hroot)
1507       . have ei : i = toLex (Sum.inl k) :=
1508         ofLex.injective (by simp using hi)
1509       have ej : j = toLex (Sum.inr l) :=
1510         ofLex.injective (by simp using hj)
1511       subst i
1512       subst j
1513       change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
1514         Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
1515       exact Sum.noConfusion hroot
1516       . have ei : i = toLex (Sum.inr k) :=
1517         ofLex.injective (by simp using hi)
1518       have ej : j = toLex (Sum.inl l) :=
1519         ofLex.injective (by simp using hj)
1520       subst i
1521       subst j
1522       change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
1523         Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
1524       exact Sum.noConfusion hroot
1525       . have ei : i = toLex (Sum.inr k) :=
1526         ofLex.injective (by simp using hi)

```

```

1527     have ej : j = toLex (Sum.inr l) :=
1528       ofLex.injective (by simp using hj)
1529     subst i
1530     subst j
1531     have hkl : k != 1 := by
1532       intro e
1533       subst l
1534       exact hij rfl
1535     change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
1536       Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
1537     exact Y.disjoint _ k l hkl a b (Sum.inr.inj hroot)
1538
1539 /-!
1540 ### The omega_0-completion used by horizontal constructions
1541
1542 The current manuscript first constructs the universal arrow over the sum of
1543 the two targets, then precomposes it with the summed origin. Its associator is
1544 obtained by flattening the corresponding universal arrow. The definitions
1545 below are the concrete Lean model of those two steps.
1546
1547 An ordinary finite folio is extended to an eventually constant omega_0-chain:
1548 page `n` is its listed page when `n < |X|`, and otherwise is its final page.
1549 Thus the extension creates no page data; it only repeats the final territories
1550 needed to make every later access well typed. A horizontal object is then
1551 stored as a flat finite family of these completed atlas components. Horizontal
1552 sum concatenates the families, while the associator merely rebrackets their
1553 sum indices and is the identity on every component. This is precisely the
1554 flattening stipulated in the preprint. Flat storage is also what prevents the
1555 finite presentation lengths and first-page partitions from depending on the
1556 parenthesization of an iterated horizontal sum.
1557 -/
1558
1559 def Folio.omegaIndex (W : Folio) (n : Nat) : Fin W.length :=
1560   W.paddedIndex n
1561
1562 theorem Folio.omegaIndex_of_lt (W : Folio) {n : Nat} (h : n < W.length) :
1563   W.omegaIndex n = <|n, h|> := by
1564   apply Fin.ext
1565   change min n (W.length - 1) = n
1566   exact Nat.min_eq_left (by omega)
1567
1568 theorem Folio.omegaIndex_of_ge (W : Folio) {n : Nat} (h : W.length <= n) :
1569   W.omegaIndex n = W.lastIndex := by
1570   apply Fin.ext
1571   change min n (W.length - 1) = W.length - 1
1572   exact Nat.min_eq_right (by omega)
1573
1574 /-- `N` is a cutoff when page `N - 1` is final and every page from `N`
1575 onward normalizes to it. -/
1576 def Folio.IsCutoff (W : Folio) (N : Nat) : Prop :=
1577   0 < N /\ forall n, N <= n -> W.omegaIndex n = W.omegaIndex (N - 1)
1578
1579 theorem Folio.length_isCutoff (W : Folio) : W.IsCutoff W.length := by
1580   refine <|W.positive, ?_|>
1581   intro n hn
1582   rw [W.omegaIndex_of_ge hn]
1583   apply Fin.ext
1584   change W.length - 1 = min (W.length - 1) (W.length - 1)
1585   simp
1586
1587 /-- The stored cardinality really is the least positive stabilization cutoff,
1588 not merely some bound after which repetition happens. -/
1589 theorem Folio.length_le_of_isCutoff (W : Folio) {N : Nat}
1590   (hN : W.IsCutoff N) : W.length <= N := by
1591   by_contra h
1592   have hN1 : N < W.length := Nat.lt_of_not_ge h
1593   have hpos := hN.1
1594   have he := hN.2 N le_rfl
1595   rw [W.omegaIndex_of_lt hN1] at he
1596   have hpred : N - 1 < W.length := lt_of_le_of_lt (Nat.sub_le N 1) hN1
1597   rw [W.omegaIndex_of_lt hpred] at he
1598   have hv : N = N - 1 := Fin.ext_iff.mp he
1599   omega

```

```

1600
1601 /-- On the repeated tail, every spine arrow commutes with the canonical
1602 identifications of its endpoints with the final finite index. -/
1603 theorem Folio.spineBase_map_tail (W : Folio) {m n : Nat}
1604   (hm : W.length <= m) (hn : W.length <= n) (f : m --> n) :
1605     W.spineBase.map f >> eqToHom (W.omegaIndex_of_ge hn) =
1606     eqToHom (W.omegaIndex_of_ge hm) := by
1607     apply Subsingleton.elim
1608
1609 /-- The corresponding coherence condition in `CoCon`: after identifying both
1610 repeated pages with the final page, the transition is the identity transition.
1611 This is the formal content of the manuscript's repeated-page clause. -/
1612 theorem Folio.F_map_tail (W : Folio) {m n : Nat}
1613   (hm : W.length <= m) (hn : W.length <= n) (f : m --> n) :
1614     W.F.map f >> W.core.map (eqToHom (W.omegaIndex_of_ge hn)) =
1615     W.core.map (eqToHom (W.omegaIndex_of_ge hm)) := by
1616     change W.core.map (W.spineBase.map f) >> _ = _
1617     rw [← W.core.map_comp]
1618     congr 1
1619
1620 def Folio.omegaBase (W : Folio) : Nat ==> Fin W.length where
1621   obj n := W.omegaIndex n
1622   map f := homOfLE (W.paddedIndex_mono (leOfHom f))
1623   map_id _ := Subsingleton.elim _ _
1624   map_comp _ _ := Subsingleton.elim _ _
1625
1626 structure OmegaFolio where
1627   finite : Folio
1628
1629 def OmegaFolio.F (W : OmegaFolio) : Nat ==> CoCon :=
1630   W.finite.F
1631
1632 def OmegaFolio.H (W : OmegaFolio) : Natop ==> Type :=
1633   W.F.leftOp then Tra
1634
1635 def OmegaFolio.originBase (_W : OmegaFolio) : Natop := op 0
1636
1637 def OmegaFolio.lastRepeatedIndex (W : OmegaFolio) (n : Nat)
1638   (h : W.finite.length <= n) :
1639   W.finite.omegaIndex n = W.finite.lastIndex :=
1640   W.finite.omegaIndex_of_ge h
1641
1642 theorem OmegaFolio.cardinality_is_least_cutoff (W : OmegaFolio) :
1643   W.finite.IsCutoff W.finite.length /\
1644     forall N, W.finite.IsCutoff N -> W.finite.length <= N :=
1645     <|W.finite.length_isCutoff, fun _ h => W.finite.length_le_of_isCutoff h|>
1646
1647 /-- A wrapper is used instead of the raw dependent sum so that Mathlib's
1648 unrelated category instance for sigma-types cannot compete with the category
1649 of elements instance. -/
1650 structure OmegaElement (W : OmegaFolio) where
1651   base : Natop
1652   value : W.H.obj base
1653
1654 instance (W : OmegaFolio) : Category (OmegaElement W) where
1655   Hom x y := { f : x.base --> y.base // W.H.map f x.value = y.value }
1656   id x := <|Id _, by simp|>
1657   comp f g := <|f.1 >> g.1, by simp [f.2, g.2]|>
1658   id_comp _ := rfl
1659   comp_id _ := rfl
1660   assoc _ _ _ := rfl
1661
1662 instance (W : OmegaFolio) (x y : OmegaElement W) : Subsingleton (x --> y) where
1663   allEq _ _ := Subtype.ext (Subsingleton.elim _ _)
1664
1665 def OmegaFolio.toFiniteElement (W : OmegaFolio) :
1666   OmegaElement W ==> W.finite.E where
1667   obj x := <|op (W.finite.omegaIndex x.base.unop), x.value|>
1668   map { _ _ } f := CategoryOfElements.homMk _ _
1669     (W.finite.omegaBase.map (Quiver.Hom.unop f.1)).op f.2
1670   map_id _ := Subsingleton.elim _ _
1671   map_comp _ _ := Subsingleton.elim _ _
1672

```

```

1673 /-- The occurrence of a final-page cell on a repeated spine page. -/
1674 def OmegaFolio.repeatElement (W : OmegaFolio) (n : Nat)
1675   (h : W.finite.length <= n)
1676   (k : W.finite.H.obj W.finite.lastBase) : OmegaElement W := by
1677   refine <|op n, ?_||>
1678   change (W.finite.core.obj (W.finite.omegaIndex n)).unop.Obj
1679   rw [W.finite.omegaIndex_of_ge h]
1680   exact k
1681
1682 structure OmegaAtl where
1683   finite : Atl
1684
1685 def OmegaAtl.folio (X : OmegaAtl) : OmegaFolio := <|X.finite.P.folio|>
1686 abbrev OmegaAtl.E (X : OmegaAtl) := OmegaElement X.folio
1687 instance (X : OmegaAtl) : Category X.E := inferInstance
1688 def OmegaAtl.G (X : OmegaAtl) : X.E ==> DomIns :=
1689   X.folio.toFiniteElement then X.finite.G
1690
1691 def OmegaAtl.originElement (X : OmegaAtl) : X.E := by
1692   refine <|op 0, ?_||>
1693   change (X.finite.P.folio.core.obj
1694     (X.finite.P.folio.omegaIndex 0)).unop.Obj
1695   rw [X.finite.P.folio.omegaIndex_of_lt X.finite.P.folio.positive]
1696   exact X.finite.P.folio.originValue
1697
1698 def OmegaAtl.extent (X : OmegaAtl) : DomIns := X.G.obj X.originElement
1699
1700 def OmegaAtl.cardinality (X : OmegaAtl) : Nat := X.finite.P.folio.length
1701
1702 theorem OmegaAtl.spine_orderType (_X : OmegaAtl) :
1703   Ordinal.type (fun m n : Nat => m < n) = Ordinal.omega0 := by
1704   change typeLT Nat = Ordinal.omega0
1705   exact Ordinal.type_nat_lt
1706
1707 def OmegaAtl.pageIndex (X : OmegaAtl) (n : Nat) : Fin X.finite.P.folio.length :=
1708   X.finite.P.folio.omegaIndex n
1709
1710 theorem OmegaAtl.pageIndex_inside (X : OmegaAtl) {n : Nat}
1711   (h : n < X.finite.P.folio.length) :
1712   X.pageIndex n = <|n, h|> :=
1713   X.finite.P.folio.omegaIndex_of_lt h
1714
1715 theorem OmegaAtl.pageIndex_outside (X : OmegaAtl) {n : Nat}
1716   (h : X.finite.P.folio.length <= n) :
1717   X.pageIndex n = X.finite.P.folio.lastIndex :=
1718   X.finite.P.folio.omegaIndex_of_ge h
1719
1720 /-- The partition condition for the completed atlas, stated at an arbitrary
1721 natural-number page. Beyond the finite presentation this is literally the
1722 partition condition of the final territory. -/
1723 def IsOmegaPagewiseDisjoint (X : OmegaAtl) : Prop :=
1724   forall (n : Nat) (i j : X.folio.H.obj (op n)), i != j ->
1725     forall (x : X.finite.G.obj
1726       (X.finite.P.cell (op (X.pageIndex n)) i))
1727       (y : X.finite.G.obj
1728         (X.finite.P.cell (op (X.pageIndex n)) j)),
1729       X.finite.G.map
1730         (X.finite.P.cellToOrigin (op (X.pageIndex n)) i) x !=
1731         X.finite.G.map
1732         (X.finite.P.cellToOrigin (op (X.pageIndex n)) j) y
1733
1734 theorem OmegaAtl.disjoint (X : OmegaAtl) : IsOmegaPagewiseDisjoint X := by
1735   intro n i j hij x y
1736   exact X.finite.disjoint (op (X.pageIndex n)) i j hij x y
1737
1738 /-- Morphisms are stored on the least finite presentations and extended
1739 canonically to the repeated spine. Consequently they cannot make independent
1740 choices on pages above the source cardinality; this is precisely the tail
1741 condition in the manuscript. -/
1742 abbrev OmegaAtlHom (X Y : OmegaAtl) := X.finite --> Y.finite
1743
1744 instance : Category OmegaAtl where
1745   Hom := OmegaAtlHom

```

```

1746 id X := Id X.finite
1747 comp f g := f >> g
1748 id_comp := Category.id_comp
1749 comp_id := Category.comp_id
1750 assoc := Category.assoc
1751
1752 def omegaCompleteObj (X : Atl) : OmegaAtl := <|X|>
1753
1754 theorem omegaComplete_eventually_territory (X : Atl) (n : Nat)
1755   (h : X.P.folio.length <= n) :
1756   X.P.folio.omegaIndex n = X.P.folio.lastIndex :=
1757   X.P.folio.omegaIndex_of_ge h
1758
1759 theorem omegaComplete_cardinality (X : Atl) :
1760   (omegaCompleteObj X).cardinality = cardinality X := rfl
1761
1762 /-!
1763 `OmegaAtlFamily` is the flat normal form used for coherence. A singleton
1764 family is an ordinary atlas after omega_0-completion; the empty family is the
1765 horizontal unit; and tensoring concatenates families. In particular, neither
1766 parenthesization creates an intermediate combined root as a new component.
1767 -/
1768
1769 structure OmegaAtlFamily where
1770   Index : Type
1771   finiteIndex : Fintype Index
1772   component : Index -> OmegaAtl
1773
1774 attribute [instance] OmegaAtlFamily.finiteIndex
1775
1776 structure OmegaAtlFamilyHom (X Y : OmegaAtlFamily) where
1777   index : X.Index -> Y.Index
1778   component : forall i, X.component i --> Y.component (index i)
1779
1780 @[ext]
1781 theorem OmegaAtlFamilyHom.ext {X Y : OmegaAtlFamily}
1782   (f g : OmegaAtlFamilyHom X Y) (hi : f.index = g.index)
1783   (hc : forall i, HEq (f.component i) (g.component i)) : f = g := by
1784   cases f
1785   cases g
1786   cases hi
1787   congr
1788   funext i
1789   exact eq_of_heq (hc i)
1790
1791 def OmegaAtlFamilyHom.identity (X : OmegaAtlFamily) :
1792   OmegaAtlFamilyHom X X where
1793   index := id
1794   component := fun _ => Id _
1795
1796 def OmegaAtlFamilyHom.comp {X Y Z : OmegaAtlFamily}
1797   (f : OmegaAtlFamilyHom X Y) (g : OmegaAtlFamilyHom Y Z) :
1798   OmegaAtlFamilyHom X Z where
1799   index := g.index o f.index
1800   component := fun i => f.component i >> g.component (f.index i)
1801
1802 instance : Category OmegaAtlFamily where
1803   Hom := OmegaAtlFamilyHom
1804   id := OmegaAtlFamilyHom.identity
1805   comp := OmegaAtlFamilyHom.comp
1806   id_comp f := by
1807     apply OmegaAtlFamilyHom.ext
1808     . rfl
1809     . intro i
1810     exact heq_of_eq (Category.id_comp _)
1811   comp_id f := by
1812     apply OmegaAtlFamilyHom.ext
1813     . rfl
1814     . intro i
1815     exact heq_of_eq (Category.comp_id _)
1816   assoc f g h := by
1817     apply OmegaAtlFamilyHom.ext
1818     . rfl

```

```

1819   . intro i
1820   exact heq_of_eq (Category.assoc _ _ _)
1821
1822 @[simp]
1823 theorem OmegaAtlFamilyHom.component_id (X : OmegaAtlFamily) (i : X.Index) :
1824   (Id X : X --> X).component i = Id (X.component i) := rfl
1825
1826 @[simp]
1827 theorem OmegaAtlFamilyHom.component_comp {X Y Z : OmegaAtlFamily}
1828   (f : X --> Y) (g : Y --> Z) (i : X.Index) :
1829   (f >> g).component i = f.component i >> g.component (f.index i) := rfl
1830
1831 def omegaAtom (X : Atl) : OmegaAtlFamily where
1832   Index := Unit
1833   finiteIndex := inferInstance
1834   component := fun _ => omegaCompleteObj X
1835
1836 def omegaFamilyUnit : OmegaAtlFamily where
1837   Index := Empty
1838   finiteIndex := inferInstance
1839   component := fun i => nomatch i
1840
1841 def omegaFamilyTensorObj (X Y : OmegaAtlFamily) : OmegaAtlFamily where
1842   Index := Sum X.Index Y.Index
1843   finiteIndex := inferInstance
1844   component := Sum.elim X.component Y.component
1845
1846 def omegaFamilyTensorHom {X_1 X_2 Y_1 Y_2 : OmegaAtlFamily}
1847   (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
1848   omegaFamilyTensorObj X_1 X_2 --> omegaFamilyTensorObj Y_1 Y_2 where
1849   index := Sum.map f.index g.index
1850   component
1851   | .inl i => f.component i
1852   | .inr j => g.component j
1853
1854 def omegaFamilyTensor : OmegaAtlFamily * OmegaAtlFamily ==> OmegaAtlFamily where
1855   obj X := omegaFamilyTensorObj X.1 X.2
1856   map f := omegaFamilyTensorHom f.1 f.2
1857   map_id X := by
1858     have hi : (omegaFamilyTensorHom (Id X.1) (Id X.2)).index =
1859       (OmegaAtlFamilyHom.identity _).index := by
1860       funext i
1861       cases i <;> rfl
1862     apply OmegaAtlFamilyHom.ext _ _ hi
1863     intro i
1864     cases i <;> exact HEq.rfl
1865   map_comp f g := by
1866     have hi : (omegaFamilyTensorHom (f.1 >> g.1) (f.2 >> g.2)).index =
1867       (omegaFamilyTensorHom f.1 f.2 >>
1868         omegaFamilyTensorHom g.1 g.2).index := by
1869       funext i
1870       cases i <;> rfl
1871     apply OmegaAtlFamilyHom.ext _ _ hi
1872     intro i
1873     cases i <;> exact HEq.rfl
1874
1875 def omegaFamilyAssociatorHom (X Y Z : OmegaAtlFamily) :
1876   omegaFamilyTensorObj (omegaFamilyTensorObj X Y) Z -->
1877   omegaFamilyTensorObj X (omegaFamilyTensorObj Y Z) where
1878   index
1879   | .inl (.inl i) => .inl i
1880   | .inl (.inr j) => .inr (.inl j)
1881   | .inr k => .inr (.inr k)
1882   component
1883   | .inl (.inl i) => Id (X.component i)
1884   | .inl (.inr j) => Id (Y.component j)
1885   | .inr k => Id (Z.component k)
1886
1887 def omegaFamilyAssociatorInv (X Y Z : OmegaAtlFamily) :
1888   omegaFamilyTensorObj X (omegaFamilyTensorObj Y Z) -->
1889   omegaFamilyTensorObj (omegaFamilyTensorObj X Y) Z where
1890   index
1891   | .inl i => .inl (.inl i)

```

```

1892 | .inr (.inl j) => .inl (.inr j)
1893 | .inr (.inr k) => .inr k
1894 component
1895 | .inl i => Id (X.component i)
1896 | .inr (.inl j) => Id (Y.component j)
1897 | .inr (.inr k) => Id (Z.component k)
1898
1899 def omegaFamilyAssociator (X Y Z : OmegaAtlFamily) :
1900   omegaFamilyTensorObj (omegaFamilyTensorObj X Y) Z Iso
1901   omegaFamilyTensorObj X (omegaFamilyTensorObj Y Z) where
1902 hom := omegaFamilyAssociatorHom X Y Z
1903 inv := omegaFamilyAssociatorInv X Y Z
1904 hom_inv_id := by
1905   apply OmegaAtlFamilyHom.ext
1906   . funext i
1907   rcases i with (i | j) | k <; rfl
1908   . intro i
1909   rcases i with (i | j) | k
1910   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1911     omegaFamilyTensorObj]
1912   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1913     omegaFamilyTensorObj]
1914   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1915     omegaFamilyTensorObj]
1916 inv_hom_id := by
1917   apply OmegaAtlFamilyHom.ext
1918   . funext i
1919   rcases i with i | (j | k) <; rfl
1920   . intro i
1921   rcases i with i | (j | k)
1922   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1923     omegaFamilyTensorObj]
1924   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1925     omegaFamilyTensorObj]
1926   . simp [omegaFamilyAssociatorHom, omegaFamilyAssociatorInv,
1927     omegaFamilyTensorObj]
1928
1929 def omegaFamilyLeftUnitorHom (X : OmegaAtlFamily) :
1930   omegaFamilyTensorObj omegaFamilyUnit X --> X where
1931 index
1932 | .inr i => i
1933 component
1934 | .inr i => Id (X.component i)
1935
1936 def omegaFamilyLeftUnitorInv (X : OmegaAtlFamily) :
1937   X --> omegaFamilyTensorObj omegaFamilyUnit X where
1938 index i := .inr i
1939 component i := Id (X.component i)
1940
1941 def omegaFamilyLeftUnitor (X : OmegaAtlFamily) :
1942   omegaFamilyTensorObj omegaFamilyUnit X Iso X where
1943 hom := omegaFamilyLeftUnitorHom X
1944 inv := omegaFamilyLeftUnitorInv X
1945 hom_inv_id := by
1946   apply OmegaAtlFamilyHom.ext
1947   . funext i
1948   rcases i with i | i
1949   . exact i.elim
1950   . rfl
1951   . intro i
1952   rcases i with i | i
1953   . exact i.elim
1954   . simp [omegaFamilyLeftUnitorHom, omegaFamilyLeftUnitorInv,
1955     omegaFamilyTensorObj]
1956 inv_hom_id := by
1957   apply OmegaAtlFamilyHom.ext
1958   . rfl
1959   . intro i
1960   simp [omegaFamilyLeftUnitorHom, omegaFamilyLeftUnitorInv,
1961     omegaFamilyTensorObj]
1962
1963 def omegaFamilyRightUnitorHom (X : OmegaAtlFamily) :
1964   omegaFamilyTensorObj X omegaFamilyUnit --> X where

```



```

1965   index
1966   | .inl i => i
1967   component
1968   | .inl i => Id (X.component i)
1969
1970 def omegaFamilyRightUnitorInv (X : OmegaAtlFamily) :
1971   X --> omegaFamilyTensorObj X omegaFamilyUnit where
1972   index i := .inl i
1973   component i := Id (X.component i)
1974
1975 def omegaFamilyRightUnitor (X : OmegaAtlFamily) :
1976   omegaFamilyTensorObj X omegaFamilyUnit Iso X where
1977   hom := omegaFamilyRightUnitorHom X
1978   inv := omegaFamilyRightUnitorInv X
1979   hom_inv_id := by
1980     apply OmegaAtlFamilyHom.ext
1981     . funext i
1982     rcases i with i | i
1983     . rfl
1984     . exact i.elim
1985   . intro i
1986     rcases i with i | i
1987     . simp [omegaFamilyRightUnitorHom, omegaFamilyRightUnitorInv,
1988       omegaFamilyTensorObj]
1989     . exact i.elim
1990   inv_hom_id := by
1991     apply OmegaAtlFamilyHom.ext
1992     . rfl
1993     . intro i
1994     simp [omegaFamilyRightUnitorHom, omegaFamilyRightUnitorInv,
1995       omegaFamilyTensorObj]
1996
1997 def omegaFamilyBraidingHom (X Y : OmegaAtlFamily) :
1998   omegaFamilyTensorObj X Y --> omegaFamilyTensorObj Y X where
1999   index
2000   | .inl i => .inr i
2001   | .inr j => .inl j
2002   component
2003   | .inl i => Id (X.component i)
2004   | .inr j => Id (Y.component j)
2005
2006 def omegaFamilyBraiding (X Y : OmegaAtlFamily) :
2007   omegaFamilyTensorObj X Y Iso omegaFamilyTensorObj Y X where
2008   hom := omegaFamilyBraidingHom X Y
2009   inv := omegaFamilyBraidingHom Y X
2010   hom_inv_id := by
2011     apply OmegaAtlFamilyHom.ext
2012     . funext i
2013     cases i <,> rfl
2014     . intro i
2015     cases i with
2016     | inl i => simp [omegaFamilyBraidingHom, omegaFamilyTensorObj]
2017     | inr j => simp [omegaFamilyBraidingHom, omegaFamilyTensorObj]
2018   inv_hom_id := by
2019     apply OmegaAtlFamilyHom.ext
2020     . funext i
2021     cases i <,> rfl
2022     . intro i
2023     cases i with
2024     | inl j => simp [omegaFamilyBraidingHom, omegaFamilyTensorObj]
2025     | inr i => simp [omegaFamilyBraidingHom, omegaFamilyTensorObj]
2026
2027 instance omegaFamilyMonoidalStruct : MonoidalCategoryStruct OmegaAtlFamily where
2028   tensorObj := omegaFamilyTensorObj
2029   tensorHom := omegaFamilyTensorHom
2030   whiskerLeft X _ f := omegaFamilyTensorHom (Id X) f
2031   whiskerRight f Y := omegaFamilyTensorHom f (Id Y)
2032   tensorUnit := omegaFamilyUnit
2033   associator := omegaFamilyAssociator
2034   leftUnitor := omegaFamilyLeftUnitor
2035   rightUnitor := omegaFamilyRightUnitor
2036
2037 instance omegaFamilyMonoidal : MonoidalCategory OmegaAtlFamily :=

```

```

2038 MonoidalCategory.ofTensorHom
2039   (id_tensorHom_id := fun X Y => by
2040     exact omegaFamilyTensor.map_id (X, Y))
2041   (id_tensorHom := fun X { _ } f => rfl)
2042   (tensorHom_id := fun { _ } f Y => rfl)
2043   (tensorHom_comp_tensorHom := fun {X_1 Y_1 Z_1 X_2 Y_2 Z_2} f_1 f_2 g_1 g_2 => by
2044     apply OmegaAt1FamilyHom.ext
2045     . funext i
2046     . cases i <=> rfl
2047     . intro i
2048     . cases i <=> simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom])
2049   (associator_naturality := fun { _ _ _ _ } f_1 f_2 f_3 => by
2050     apply OmegaAt1FamilyHom.ext
2051     . funext i
2052     . rcases i with (i | j) | k <=> rfl
2053     . intro i
2054     . rcases i with (i | j) | k <=>
2055       simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyAssociator,
2056         omegaFamilyAssociatorHom, omegaFamilyTensorObj])
2057   (leftUnitor_naturality := fun { _ } f => by
2058     apply OmegaAt1FamilyHom.ext
2059     . funext i
2060     . rcases i with i | i
2061     . exact i.elim
2062     . rfl
2063     . intro i
2064     . rcases i with i | i
2065     . exact i.elim
2066     . simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyLeftUnitor,
2067       omegaFamilyLeftUnitorHom, omegaFamilyTensorObj])
2068   (rightUnitor_naturality := fun { _ } f => by
2069     apply OmegaAt1FamilyHom.ext
2070     . funext i
2071     . rcases i with i | i
2072     . rfl
2073     . exact i.elim
2074     . intro i
2075     . rcases i with i | i
2076     . simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyRightUnitor,
2077       omegaFamilyRightUnitorHom, omegaFamilyTensorObj]
2078     . exact i.elim)
2079   (pentagon := fun W X Y Z => by
2080     apply OmegaAt1FamilyHom.ext
2081     . funext i
2082     . rcases i with ((i | j) | k) | l <=> rfl
2083     . intro i
2084     . rcases i with ((i | j) | k) | l <=>
2085       simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyAssociator,
2086         omegaFamilyAssociatorHom, omegaFamilyTensorObj])
2087   (triangle := fun X Y => by
2088     apply OmegaAt1FamilyHom.ext
2089     . funext i
2090     . rcases i with (i | e) | j
2091     . rfl
2092     . exact e.elim
2093     . rfl
2094     . intro i
2095     . rcases i with (i | e) | j
2096     . simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyAssociator,
2097       omegaFamilyAssociatorHom, omegaFamilyRightUnitor,
2098       omegaFamilyRightUnitorHom, omegaFamilyTensorObj]
2099     . exact e.elim
2100     . simp [omegaFamilyMonoidalStruct, omegaFamilyTensorHom, omegaFamilyAssociator,
2101       omegaFamilyAssociatorHom, omegaFamilyLeftUnitor,
2102       omegaFamilyLeftUnitorHom, omegaFamilyTensorObj])
2103
2104 @[simp]
2105 theorem omegaFamily_tensorObj (X Y : OmegaAt1Family) :
2106   MonoidalCategoryStruct.tensorObj X Y = omegaFamilyTensorObj X Y := rfl
2107
2108 @[simp]
2109 theorem omegaFamily_tensorHom {X_1 Y_1 X_2 Y_2 : OmegaAt1Family}
2110   (f : X_1 --> Y_1) (g : X_2 --> Y_2) :

```

```

2111 MonoidalCategoryStruct.tensorHom f g = omegaFamilyTensorHom f g := rfl
2112
2113 @[simp]
2114 theorem omegaFamily_whiskerLeft (X : OmegaAtlFamily)
2115   {Y_1 Y_2 : OmegaAtlFamily} (f : Y_1 --> Y_2) :
2116   MonoidalCategoryStruct.whiskerLeft X f =
2117     omegaFamilyTensorHom (Id X) f := rfl
2118
2119 @[simp]
2120 theorem omegaFamily_whiskerRight {X_1 X_2 : OmegaAtlFamily}
2121   (f : X_1 --> X_2) (Y : OmegaAtlFamily) :
2122   MonoidalCategoryStruct.whiskerRight f Y =
2123     omegaFamilyTensorHom f (Id Y) := rfl
2124
2125 @[simp]
2126 theorem omegaFamily_associator (X Y Z : OmegaAtlFamily) :
2127   MonoidalCategoryStruct.associator X Y Z = omegaFamilyAssociator X Y Z := rfl
2128
2129 instance omegaFamilyBraided : BraidedCategory OmegaAtlFamily where
2130   braiding := omegaFamilyBraiding
2131   braiding_naturality_right := fun X { _ _ } f => by
2132     apply OmegaAtlFamilyHom.ext
2133     . funext i
2134       cases i <|> rfl
2135     . intro i
2136       cases i <|>
2137         simp [omegaFamilyTensorHom,
2138               omegaFamilyBraiding, omegaFamilyBraidingHom,
2139               omegaFamilyTensorObj]
2140   braiding_naturality_left := fun { _ _ } f Z => by
2141     apply OmegaAtlFamilyHom.ext
2142     . funext i
2143       cases i <|> rfl
2144     . intro i
2145       cases i <|>
2146         simp [omegaFamilyTensorHom,
2147               omegaFamilyBraiding, omegaFamilyBraidingHom,
2148               omegaFamilyTensorObj]
2149   hexagon_forward := fun X Y Z => by
2150     apply OmegaAtlFamilyHom.ext
2151     . funext i
2152       rcases i with (i | j) | k <|> rfl
2153     . intro i
2154       rcases i with (i | j) | k <|>
2155         simp [omegaFamilyTensorHom,
2156               omegaFamilyAssociator, omegaFamilyAssociatorHom,
2157               omegaFamilyBraiding, omegaFamilyBraidingHom,
2158               omegaFamilyTensorObj]
2159   hexagon_reverse := fun X Y Z => by
2160     apply OmegaAtlFamilyHom.ext
2161     . funext i
2162       rcases i with i | (j | k) <|> rfl
2163     . intro i
2164       rcases i with i | (j | k) <|>
2165         simp [omegaFamilyTensorHom,
2166               omegaFamilyAssociator, omegaFamilyAssociatorInv,
2167               omegaFamilyBraiding, omegaFamilyBraidingHom,
2168               omegaFamilyTensorObj]
2169
2170 instance : SymmetricCategory OmegaAtlFamily where
2171   symmetry X Y := by
2172     apply OmegaAtlFamilyHom.ext
2173     . funext i
2174       cases i <|> rfl
2175     . intro i
2176       cases i <|>
2177         simp [omegaFamilyBraided, omegaFamilyBraiding,
2178               omegaFamilyBraidingHom, omegaFamilyTensorObj]
2179
2180 /-- Implementation note: the universal construction is represented by flat
2181 finite families of atlases completed along the spine. Keeping this note out
2182 of the manuscript block ensures that the compiled definition remains the one
2183 stated in `datra.md`. -/

```

```

2184 abbrev AtlHor := OmegaAtlFamily
2185
2186 def AtlHorSum : CategoryTheory.Functor (AtlHor * AtlHor) AtlHor :=
2187   omegaFamilyTensor
2188
2189 /-- Implementation note: on arrows, horizontal sum performs the simultaneous
2190 precomposition from the two summands of the source. The two cases below are
2191 the formal counterparts of the canonical inclusions of `X` and `X'` into the
2192 origin `X + X'` in the manuscript's universal-arrow presentation. -/
2193 theorem AtlHorSum_map {X X' Y Y' : AtlHor} (F : X --> Y) (F' : X' --> Y') :
2194   AtlHorSum.map ((F, F') : (X, X') --> (Y, Y')) =
2195     omegaFamilyTensorHom F F' := rfl
2196
2197 theorem horizontalLemma : Nonempty (SymmetricCategory AtlHor) :=
2198   <|inferInstance|>
2199
2200 def AtlBrd (X Y : AtlHor) :
2201   AtlHorSum.obj (X, Y) Iso AtlHorSum.obj (Y, X) :=
2202     omegaFamilyBraiding X Y
2203
2204 def AtlAsoc (X Y Z : AtlHor) :
2205   AtlHorSum.obj (AtlHorSum.obj (X, Y), Z) Iso
2206     AtlHorSum.obj (X, AtlHorSum.obj (Y, Z)) :=
2207     omegaFamilyAssociator X Y Z
2208
2209 /-- Flattening the universal arrow introduces no morphism inside an atlas: it
2210 only reassociates the three summand indices, and every component map is an
2211 identity. This is the implementation-level content of the final sentence in
2212 the manuscript's definition of `AtlAsoc`. -/
2213 theorem AtlAsoc_component (X Y Z : AtlHor)
2214   (i : (AtlHorSum.obj (AtlHorSum.obj (X, Y), Z)).Index) :
2215     HEq ((AtlAsoc X Y Z).hom.component i)
2216       (Id ((AtlHorSum.obj (AtlHorSum.obj (X, Y), Z)).component i)) := by
2217       rcases i with (i | j) | k <;> exact HEq.rfl
2218
2219 /-- The atlas left unitor, with the standard source `I tensor X`. -/
2220 def AtlLu (X : AtlHor) : AtlHorSum.obj (omegaFamilyUnit, X) Iso X :=
2221   omegaFamilyLeftUnitor X
2222
2223 /-- The atlas right unitor, with the standard source `X tensor I`. -/
2224 def AtlRu (X : AtlHor) : AtlHorSum.obj (X, omegaFamilyUnit) Iso X :=
2225   omegaFamilyRightUnitor X
2226
2227 instance : SymmetricCategory OmegaAtlFamilyop where
2228   symmetry X Y := by
2229     apply Quiver.Hom.unop_inj
2230     simp
2231
2232 /-! The completed presheaf category on which Day convolution is formed. The
2233 all-objects full subcategory is deliberate: it gives the Day convolution a
2234 fresh monoidal instance, distinct from the pointwise cartesian monoidal
2235 structure that the raw functor category already carries. -/
2236 abbrev OmegaPresheaf :=
2237   CategoryTheory.Functor OmegaAtlFamilyop (Type 3)
2238
2239 def IsOmegaDaTra : ObjectProperty OmegaPresheaf := fun _ => True
2240
2241 abbrev OmegaDaTra := IsOmegaDaTra.FullSubcategory
2242
2243 abbrev omegaDaTraInc : CategoryTheory.Functor OmegaDaTra OmegaPresheaf :=
2244   IsOmegaDaTra.iota
2245
2246 theorem omegaDaTraInc_essImage (F : OmegaPresheaf) :
2247   omegaDaTraInc.essImage F :=
2248     <|<|F, trivial|>, <|Iso.refl _|>|>
2249
2250 instance omegaPreservesTensorRightForTensor (v : Type 3)
2251   (d : OmegaAtlFamilyop) :
2252     PreservesColimitsOfShape
2253       (CostructuredArrow (MonoidalCategory.tensor OmegaAtlFamilyop) d)
2254       (MonoidalCategory.tensorRight v) :=
2255     preservesColimitsOfShape_of_natIso
2256       (BraidedCategory.tensorLeftIsoTensorRight v)

```

```

2257
2258 instance omegaPreservesTensorRightForUnit (v : Type 3)
2259   (d : OmegaAtlFamilyop) :
2260   PreservesColimitsOfShape
2261     (CostructuredArrow
2262       (CategoryTheory.Functor.fromPUnit
2263         (MonoidalCategory.tensorUnit OmegaAtlFamilyop)) d)
2264     (MonoidalCategory.tensorRight v) :=
2265   preservesColimitsOfShape_of_natIso
2266     (BraidedCategory.tensorLeftIsoTensorRight v)
2267
2268 instance omegaPreservesTensorRightForUnitProduct (v : Type 3)
2269   (d : OmegaAtlFamilyop * OmegaAtlFamilyop) :
2270   PreservesColimitsOfShape
2271     (CostructuredArrow
2272       ((CategoryTheory.Functor.id OmegaAtlFamilyop).prod
2273         (CategoryTheory.Functor.fromPUnit
2274           (MonoidalCategory.tensorUnit OmegaAtlFamilyop)))) d)
2275     (MonoidalCategory.tensorRight v) :=
2276   preservesColimitsOfShape_of_natIso
2277     (BraidedCategory.tensorLeftIsoTensorRight v)
2278
2279 instance omegaPreservesTensorRightForTensorProduct (v : Type 3)
2280   (d : OmegaAtlFamilyop * OmegaAtlFamilyop) :
2281   PreservesColimitsOfShape
2282     (CostructuredArrow
2283       ((MonoidalCategory.tensor OmegaAtlFamilyop).prod
2284         (CategoryTheory.Functor.id OmegaAtlFamilyop)) d)
2285     (MonoidalCategory.tensorRight v) :=
2286   preservesColimitsOfShape_of_natIso
2287     (BraidedCategory.tensorLeftIsoTensorRight v)
2288
2289 noncomputable def omegaDaTraMonoidal : MonoidalCategory OmegaDaTra :=
2290   MonoidalCategory.monoidalOfHasDayConvolutions omegaDaTraInc
2291     (ObjectProperty.fullyFaithfuliota IsOmegaDaTra)
2292     (fun _ _ => omegaDaTraInc_essImage _)
2293     (omegaDaTraInc_essImage _)
2294
2295 noncomputable instance : MonoidalCategory OmegaDaTra := omegaDaTraMonoidal
2296
2297 noncomputable instance omegaDaTraLawful :
2298   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct
2299     OmegaAtlFamilyop (Type 3) OmegaDaTra :=
2300   MonoidalCategory.lawfulDayConvolutionMonoidalCategoryStructOfHasDayConvolutions
2301     omegaDaTraInc (ObjectProperty.fullyFaithfuliota IsOmegaDaTra)
2302     (fun _ _ => omegaDaTraInc_essImage _)
2303     (omegaDaTraInc_essImage _)
2304
2305 noncomputable instance omegaDayConvolution (F G : OmegaDaTra) :
2306   MonoidalCategory.DayConvolution
2307     (omegaDaTraInc.obj F) (omegaDaTraInc.obj G) :=
2308   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution
2309     OmegaAtlFamilyop (Type 3) OmegaDaTra F G
2310
2311 noncomputable instance omegaDayConvolutionRightNested
2312   (F G H : OmegaDaTra) :
2313   MonoidalCategory.DayConvolution (omegaDaTraInc.obj F)
2314     (MonoidalCategory.DayConvolution.convolution
2315       (omegaDaTraInc.obj G) (omegaDaTraInc.obj H)) :=
2316   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2
2317     OmegaAtlFamilyop (Type 3) OmegaDaTra F G H
2318
2319 noncomputable instance omegaDayConvolutionLeftNested
2320   (F G H : OmegaDaTra) :
2321   MonoidalCategory.DayConvolution
2322     (MonoidalCategory.DayConvolution.convolution
2323       (omegaDaTraInc.obj F) (omegaDaTraInc.obj G))
2324     (omegaDaTraInc.obj H) :=
2325   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2'
2326     OmegaAtlFamilyop (Type 3) OmegaDaTra F G H
2327
2328 noncomputable def omegaDaTraBraiding (F G : OmegaDaTra) :
2329   MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F := by

```

```

2330 exact (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).preimageIso
2331   (MonoidalCategory.DayConvolution.braiding
2332     (omegaDaTraInc.obj F) (omegaDaTraInc.obj G))
2333
2334 theorem omegaDaTraInc_map_braiding_hom (F G : OmegaDaTra) :
2335   omegaDaTraInc.map (omegaDaTraBraiding F G).hom =
2336     (MonoidalCategory.DayConvolution.braiding
2337       (omegaDaTraInc.obj F) (omegaDaTraInc.obj G)).hom := by
2338   exact (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_preimage _
2339
2340 theorem omegaDaTraInc_map_braiding_inv (F G : OmegaDaTra) :
2341   omegaDaTraInc.map (omegaDaTraBraiding F G).inv =
2342     (MonoidalCategory.DayConvolution.braiding
2343       (omegaDaTraInc.obj F) (omegaDaTraInc.obj G)).inv := by
2344   exact (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_preimage _
2345
2346 theorem omegaDaTraInc_map_tensorHom
2347   {F_1 F_2 G_1 G_2 : OmegaDaTra}
2348   (f : F_1 --> F_2) (g : G_1 --> G_2) :
2349   omegaDaTraInc.map (MonoidalCategory.tensorHom f g) =
2350     MonoidalCategory.DayConvolution.map
2351       (omegaDaTraInc.map f) (omegaDaTraInc.map g) := by
2352   simpa [omegaDaTraInc, omegaDaTraLawful] using
2353     (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.iota_map_tensorHom_hom_eq_tensorHom
2354       OmegaAt1Familyop (Type 3) OmegaDaTra f g)
2355
2356 theorem omegaDaTraInc_map_associator_hom (F G H : OmegaDaTra) :
2357   omegaDaTraInc.map (MonoidalCategory.associator F G H).hom =
2358     (MonoidalCategory.DayConvolution.associator
2359       (omegaDaTraInc.obj F) (omegaDaTraInc.obj G)
2360       (omegaDaTraInc.obj H)).hom := by
2361   simpa [omegaDaTraInc, omegaDaTraLawful] using
2362     (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.
2363       iota_map_associator_hom_eq_associator_hom
2364       OmegaAt1Familyop (Type 3) OmegaDaTra F G H)
2365
2366 noncomputable instance omegaDaTraBraided : BraidedCategory OmegaDaTra where
2367   braiding := omegaDaTraBraiding
2368   braiding_naturality_right := fun X { _ _ } f => by
2369     rw [← MonoidalCategory.id_tensorHom,
2370       ← MonoidalCategory.tensorHom_id]
2371     apply (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_injective
2372     simp only [Functor.map_comp,
2373       omegaDaTraInc_map_tensorHom, omegaDaTraInc_map_braiding_hom]
2374     exact MonoidalCategory.DayConvolution.braiding_naturality_right
2375       (omegaDaTraInc.obj X) (omegaDaTraInc.map f)
2376   braiding_naturality_left := fun { _ _ } f Z => by
2377     rw [← MonoidalCategory.tensorHom_id,
2378       ← MonoidalCategory.id_tensorHom]
2379     apply (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_injective
2380     simp only [Functor.map_comp,
2381       omegaDaTraInc_map_tensorHom, omegaDaTraInc_map_braiding_hom]
2382     exact MonoidalCategory.DayConvolution.braiding_naturality_left
2383       (omegaDaTraInc.map f) (omegaDaTraInc.obj Z)
2384   hexagon_forward := by
2385     intro X Y Z
2386     apply (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_injective
2387     simp only [Functor.map_comp,
2388       omegaDaTraInc_map_associator_hom,
2389       omegaDaTraInc_map_braiding_hom]
2390     rw [← MonoidalCategory.tensorHom_id,
2391       ← MonoidalCategory.id_tensorHom]
2392     simp only [omegaDaTraInc_map_tensorHom]
2393     exact MonoidalCategory.DayConvolution.hexagon_forward
2394       (omegaDaTraInc.obj X) (omegaDaTraInc.obj Y) (omegaDaTraInc.obj Z)
2395   hexagon_reverse := by
2396     intro X Y Z
2397     apply (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_injective
2398     simp only [Functor.map_comp,
2399       omegaDaTraInc_map_braiding_hom]
2400     rw [← MonoidalCategory.id_tensorHom,
2401       ← MonoidalCategory.tensorHom_id]
2402     simp only [omegaDaTraInc_map_tensorHom]

```

```

2402     exact MonoidalCategory.DayConvolution.hexagon_reverse
2403         (omegaDaTraInc.obj X) (omegaDaTraInc.obj Y) (omegaDaTraInc.obj Z)
2404
2405 set_option maxHeartbeats 2400000 in
2406 noncomputable instance : SymmetricCategory OmegaDaTra where
2407     symmetry F G := by
2408         apply (ObjectProperty.fullyFaithfuliota IsOmegaDaTra).map_injective
2409         simp only [Functor.map_comp]
2410     exact MonoidalCategory.DayConvolution.symmetry
2411         (omegaDaTraInc.obj F) (omegaDaTraInc.obj G)
2412
2413 /-- The implementation realizes the manuscript's horizontal site by the flat
2414 completed atlas category `AtlHor`; `DaTravMon` is its full presheaf category.
2415 This completion detail is intentionally outside the extracted definition. -/
2416 abbrev DaTravMon := OmegaDaTra
2417
2418 noncomputable def I : DaTravMon :=
2419     MonoidalCategory.tensorUnit DaTravMon
2420
2421 /-- Certificate of the universal property implicit in the notation
2422 `I = Yo(AtlI)`: under the inclusion into presheaves, `I` is the pointwise left
2423 Kan extension of the singleton along the inclusion of the horizontal unit. -/
2424 noncomputable def I_dayConvolutionUnit :
2425     MonoidalCategory.DayConvolutionUnit (omegaDaTraInc.obj I) :=
2426     MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolutionUnit
2427     OmegaAtlFamilyop (Type 3) DaTravMon
2428
2429 noncomputable def HorSum :
2430     CategoryTheory.Functor (DaTravMon * DaTravMon) DaTravMon :=
2431     MonoidalCategory.tensor DaTravMon
2432
2433 noncomputable def Brd (F G : DaTravMon) :
2434     MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F :=
2435     omegaDaTraBraiding F G
2436
2437 noncomputable def Asoc (F G H : DaTravMon) :
2438     MonoidalCategory.tensorObj (MonoidalCategory.tensorObj F G) H Iso
2439     MonoidalCategory.tensorObj F (MonoidalCategory.tensorObj G H) :=
2440     MonoidalCategory.associator F G H
2441
2442 noncomputable def Lu (F : DaTravMon) :
2443     MonoidalCategory.tensorObj I F Iso F :=
2444     MonoidalCategory.leftUnitor F
2445
2446 noncomputable def Ru (F : DaTravMon) :
2447     MonoidalCategory.tensorObj F I Iso F :=
2448     MonoidalCategory.rightUnitor F
2449
2450 /-- Unlike the prose's abbreviated "can be checked", the Lean instance above
2451 contains the naturality, pentagon, triangle, two hexagons, and symmetry proofs
2452 explicitly. -/
2453 theorem serenityLemma : Nonempty (SymmetricCategory DaTravMon) :=
2454     <|inferInstance|>
2455
2456 end
2457
2458 end Datra

```

Listing 1: Complete Lean formalization of DaTra